

Introduction to Statistics for Parasitology

1 Overview

This document is an overview for some statistical methods commonly used in parasitology (and biology in general). It is not meant to be a statistical textbook, just a primer to get you up and running with your own analysis. For this tutorial, you will need a computer with a recent version of [R](#) and [RStudio installed](#). If you cannot install RStudio, perhaps because you have a Chromebook, you can use the Posit [RStudio Cloud edition](#) (requires free account signup).

Tip

Install R *before* installing RStudio. The download links at the link above should take care of this for you.

2 Why do we have to do statistics?

Science works by testing **hypotheses**, or proposed explanations for natural phenomena, against data. At some point, a scientist must determine whether the data they have collected **support** or **do not support** their hypothesis. Support or non-support manifest as the data either matching the predictions of a hypothesis, or not matching the predictions of a hypothesis. The **predictions** of a hypothesis are patterns that would be expected in the data if the hypothesis were true. In science, we require that any hypothesis be **falsifiable**, at least in principle. This means that there must be some observation that would prove a hypothesis false. That is, a false hypothesis must make at least some false predictions. A true hypothesis, by contrast, should generate *only* true predictions.

The trouble is, biological data are noisy. Individual organisms differ from one another, measurements are imperfect, and random chance influences every sample we collect. As a result, patterns can appear in a dataset even when no real biological effect exists. Our goal is to distinguish true biological **signals** from background **noise**.

Statistical tests help us do this. They ask whether the pattern we observed is stronger than we would expect to see from random variation alone. If the observed pattern would be very unlikely to arise from noise by itself, we have evidence that a real signal may be present. This evidence is summarized by the p -value, which estimates how surprising our data would be if there were actually no biological effect. Smaller p -values suggest a stronger signal relative to the background noise, while larger p -values suggest that the observed pattern could easily be explained by random variation. Patterns that are stronger than what would be expected by chance are called **statistically significant**. For historical (and somewhat arbitrary and controversial reasons), biologists usually use $p < 0.05$ as the threshold for significance.

Let's consider an example. A researcher observes that she tends to find more parasitic worms in larger fish. After some reading and thinking, she develops a hypothesis:

“Larger fish tend to have more parasites because larger fish tend to be older, and thus have had more time to accumulate parasites.”

From this proposed explanation, the researcher can deduce several predictions that logically follow:

1. **Prediction 1:** Adult fish will have more parasites in their bodies than juvenile fish.
2. **Prediction 2:** There will be a positive correlation between fish body length and the number of parasitic worms in that fish.

If both of these predictions are true, then that would be good support for her hypothesis. If one or neither is true, then the hypothesis is not supported and should either be revised or discarded.

The researcher collects data from as many fish as she can find, and gets these results. First, the results related to prediction 1:

Second, the results related to prediction 2:

Were her predictions supported? The figures suggest yes, but we need to do a formal test in order to say yes with confidence.

For the first figure, what we want is to see if the means of the two groups differ. If the means differ, then the hypothesis is supported. If the means do not differ, then the hypothesis is *not* supported. It's not enough to just look at the sample means—9.67 for adults and 4.92 for juveniles—we have to compare the pattern we observed to what could have happened if the prediction was false. That is, if the means were really the same. The proper test for this situation is called a **Student's t -test**, or just **t -test**. In this case:

```
t.test(y~age, data=dx)
```

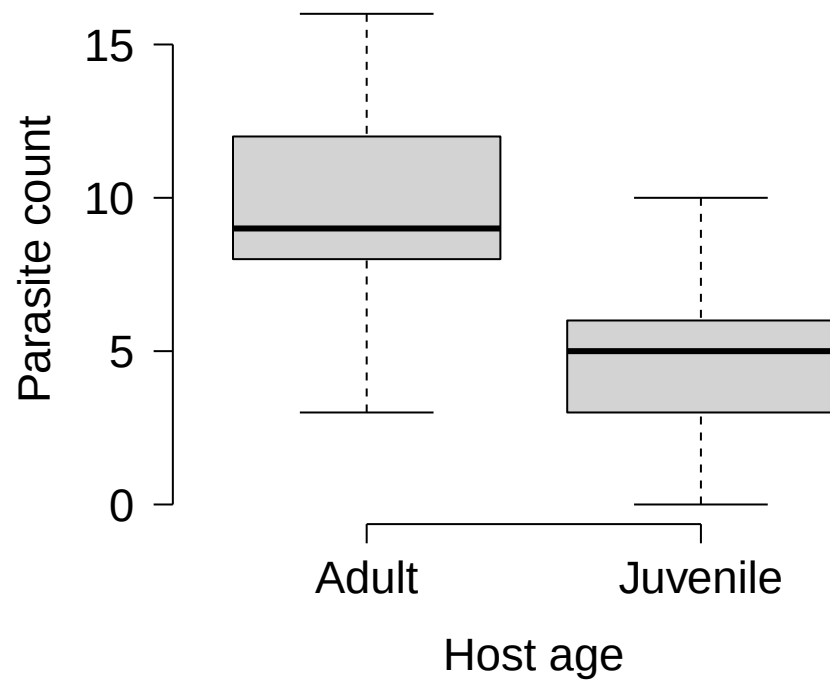


Figure 1: Example boxplot from a simulated study showing difference in parasite count between adult and juvenile fish.

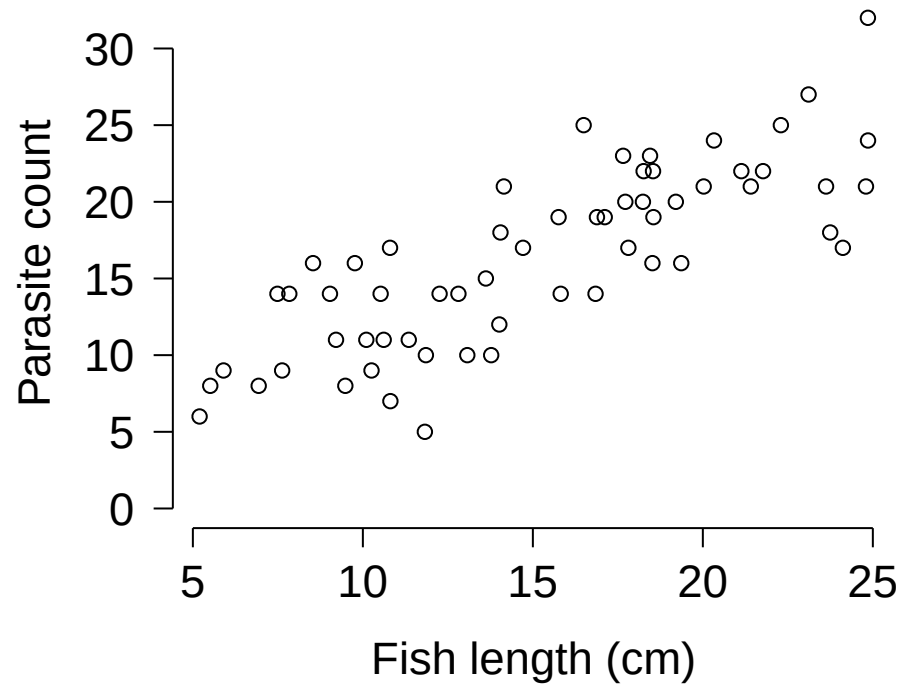


Figure 2: Example scatterplot from a simulated study showing a positive correlation between parasite count and body length in fish.

Welch Two Sample t-test

```
data: y by age
t = 13.349, df = 188.92, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Adult and group Juvenile is not equal to 0
95 percent confidence interval:
 4.048095 5.451905
sample estimates:
mean in group Adult mean in group Juvenile
          9.67          4.92
```

The p -value indicates that the difference in means between the adults and juveniles is **statistically significant**: larger and more consistent than what would be expected if there was no difference between groups.

In the case of prediction 2, the researcher could check for a **linear correlation**, or reliable covariation between the “parasite count” and “fish length”:

```
cor.test(dy$x, dy$y)
```

Pearson's product-moment correlation

```
data: dy$x and dy$y
t = 10.261, df = 58, p-value = 1.183e-14
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6897073 0.8779160
sample estimates:
cor
0.8029997
```

Again, the p -value less than 0.05 confirms the researcher's suspicion that the correlation was stronger than might be expected if there was no correlation.

These were just examples—we will explore each of these techniques in detail below. For now, just remember:

1. Scientists look for patterns, but data is noisy.
2. Statistics helps us tell how likely it is we are seeing signal or noise—but does not **prove** whether a signal exists.

3. A p -value can never be 0, so statistics can always lead us astray if we are not careful.
4. “Statistically significant” is not the same as “biologically significant”.

3 Types of tests

There are thousands of statistical tests out there, but most biologists only ever need a handful. One of the hardest parts of “doing statistics” for many people is choosing which test to use. If you ask a statistician, they are going to ask you, “Well, what *question* are you trying to answer?”. This is because every statistical test answers a particular question:

- Do groups A and B have the same mean?
- As variable x changes, does variable y also tend to change?
- If variable x changes by this much, how much will y change?
- If event G happens, how likely is it that event H also happens?
- Do values in group A tend to be greater than, or smaller than, values in group B ?
- Are there categories that consistently and reliably capture the variation in these sample units?

Notice that all of the statements above are statistical statements, not necessarily biological statements. Translating biological ideas and questions into statistical statements and questions is one of the hardest parts of doing statistics.

The hypotheses that we test in biology tend to fall into just a few categories:

1. Testing for a difference in **mean** or **location** between groups.
2. Testing for a **continuous relationship** between a **response variable** (or multiple variables) and one or more **predictor variables**. Also referred to as predicting or **modeling** a continuous variable.
3. Testing for (lack of) **independence between categories** of observations.
4. **Classifying** observations based on predictor variables.

These categories are not mutually exclusive...some questions can be analyzed in > 1 way!

4 Summary statistics

4.1 Central tendency and variability

One of the first tasks in data analysis is to summarize the data. This means presenting values that express the average or **central tendency**—the typical value in a dataset—and some idea of the **variability**.

Central tendency is usually expressed as the **mean** or **median**:

- **Mean:** the sum of all values divided by the number of values. This tells you, “if all values were the same, what would they be?”.
- **Median:** the middle value or 50th percentile. If you lined up all values from smallest to greatest, this value would be in the center.

Variability is usually expressed as the **standard deviation** or **interquartile range**:

- **Standard deviation (SD):** the square-root of the **variance**, or mean squared deviation from the mean. The larger the SD, the more spread out the values tend to be.
- **Interquartile range (IQR):** the difference between the 75th percentile and the 25th percentile of a set of observations. By definition, 50% of observations fall in this interval. The larger the IQR, the more spread out the value are.

When summarizing data, we usually present the sample mean $\bar{x}$ and the sample SD s as $\bar{x} \pm s$. E.g., 3.2 ± 0.8 cm.

We can see the effect of the mean and SD by examining some distributions with different μ and σ .

```
# possible values of distribution
xvals <- seq(-5, 5, length=1000)

# calculate probability distribution function (PDF)
# of each x value under different mu and sigma.
# PDF expresses relative likelihood of a value.
pdf1 <- dnorm(xvals, mean=0, sd=0.2)
pdf2 <- dnorm(xvals, mean=0, sd=1)
pdf3 <- dnorm(xvals, mean=0, sd=4)
pdf4 <- dnorm(xvals, mean=2, sd=0.5)

# plot the PDFs
plot(xvals, pdf1, type="l", lwd=3,
     xlab="X value", ylab="PDF(x)",
```

```

ylim=c(0, 2))
points(xvals, pdf2, type="l", lwd=3, col="red3")
points(xvals, pdf3, type="l", lwd=3, col="blue3")
points(xvals, pdf4, type="l", lwd=3, col="green4")
legend("topleft",
      legend=c(
        expression(mu==0~sigma==0.2),
        expression(mu==0~sigma==1),
        expression(mu==0~sigma==4),
        expression(mu==2~sigma==0.5)
      ),
      lwd=3, col=c("black", "red3", "blue3", "green4"))

```

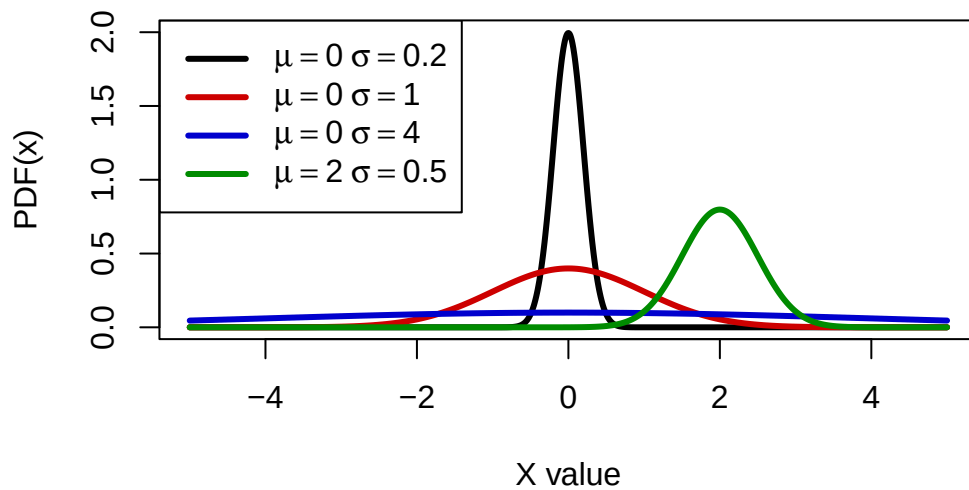


Figure 3: Illustration of normal distributions with different means and standard deviations.

The figure shows that the mean μ shifts the mean, which is also the most likely value (and thus peak of the graph). The SD σ determines how spread out the values are.

4.2 Sample vs. population statistics

When we collect data, we collect a **sample** of all possible values. The set of all possible values is the **population** of values. As biologists we are usually interested in making inferences about the population using estimates derived from sample data.

Let's explore some data from the [normal distribution](#) for an example. We will use R to draw *samples* from a distribution with a population mean (μ) of 10 and a population SD (σ) of 3.

```
# draw 10 values from the underlying distribution
rand.values <- rnorm(10, mean=10, sd=3)

# calculate the sample mean and sample SD
mean(rand.values)
```

```
[1] 9.815367
```

```
sd(rand.values)
```

```
[1] 2.794991
```

Your results will be different from those above, because the values are random, but the sample mean is likely around 10 and the sample SD around 3. If you repeat those 3 commands, you will get different numbers each time. The sample mean will probably never exactly equal the true μ , but it will be close. If you take more samples, then the sample mean should become a better estimator of the population mean:

```
# draw 100 values from the underlying distribution,
# then calculate the sample mean and SD
rand.values100 <- rnorm(100, mean=10, sd=3)
mean(rand.values100)
```

```
[1] 10.03784
```

```
sd(rand.values100)
```

```
[1] 2.773737
```

Because of randomness, your sample mean might be more correct with 10 samples than with 1000 samples! But, the long-run average of sample means will be better estimators of μ with larger sample sizes. We can test this with simulation:

```

# population parameters
mu <- 10
sigma <- 3

# sample sizes to test
samp.sizes <- c(3:10, 2:10*10, 2:10*100)
n.samp.sizes <- length(samp.sizes)

# how many simulations per sample size?
sims <- 1000

# structure to store simulated sample means
# for each sample size
sim.list <- vector("list", length=n.samp.sizes)

# execute simulations in a loop
for(i in 1:n.samp.sizes){
  # vector to hold simulated sample means for
  # each replicate j
  sim.vector <- numeric(sims)
  # simulate sample means in a loop
  for(j in 1:sims){
    sim.vector[j] <- mean(rnorm(samp.sizes[i], mu, sigma))
  }#j
  # store in list
  sim.list[[i]] <- sim.vector
}#i

# what is the mean estimate at each sample size?
mean.mean <- sapply(sim.list, mean)
# what is the spread of estimates of mu at each
# sample size?
sim.ranges <- sapply(sim.list, range)

# plot the estimate and range of estimates vs.
# sample size
plot(samp.sizes, mean.mean, lwd=3, type="o", pch=16, log="x",
     cex=1.5, xlab="Sample size", ylab="Mean of sample means",
     ylim=c(5, 15))
segments(samp.sizes, sim.ranges[1,], samp.sizes, sim.ranges[2,],
        lwd=3)

```

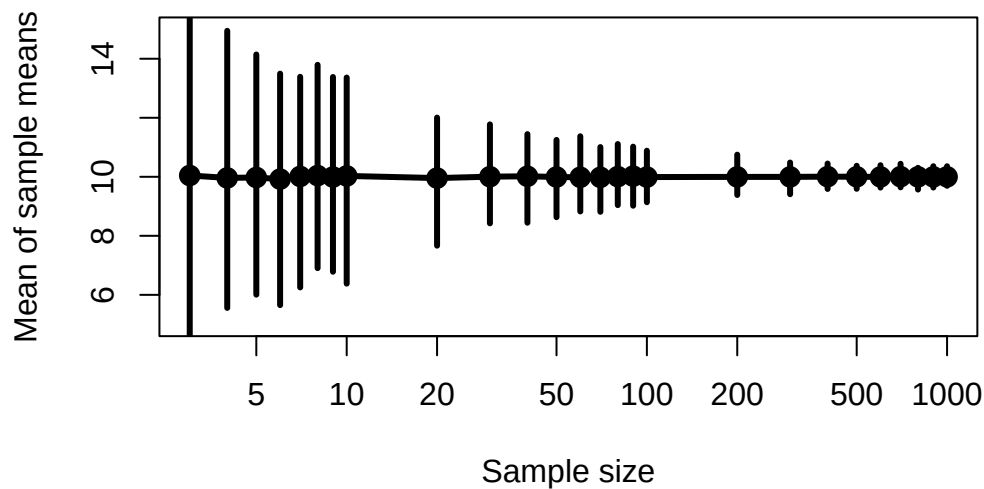


Figure 4: As sample size increases, the estimate of the population mean becomes more and more certain.

As you can see, the estimates of the true population mean converge rather quickly on the correct value. But the uncertainty around that estimate (the width of the vertical bars) takes quite a while longer to shrink. Try repeating the code block above with different values of `mu` and `sigma` to see if the pattern changes.

4.3 Summarizing data in R

The most common summary statistics have eponymous functions in R: `mean()`, `sd()`, `var()`, `IQR()`, `quantile()`, and so on.

```
# get some random values
# with mu = 0 and sigma = 1
# (standard normal distribution)
z <- rnorm(100)
```

```
# mean
mean(z)
```

```
[1] -0.01342545
```

```
# sd  
sd(z)
```

```
[1] 0.9948091
```

```
# variance  
var(z)
```

```
[1] 0.9896452
```

```
# quantiles  
## 20th percentile: 20% of values are <= this value  
quantile(z, 0.2)
```

```
20%  
-0.82015
```

```
## 90th percentile: 90% of values are <= this value  
quantile(z, 0.9)
```

```
90%  
1.247776
```

```
# median  
median(z)
```

```
[1] 0.03562525
```

```
# also the median  
quantile(z, 0.5)
```

```
50%  
0.03562525
```

```
# IQR  
IQR(z)
```

```
[1] 1.382284
```

When summarizing values, we typically write either $\bar{x} \pm s$ or *median (IQR)*. Make sure not to confuse the sample mean $\bar{x}$ with the population mean μ . Likewise, do not present the sample SD s as the population SD σ .

 Tip

Missing values

When a dataset contains missing values, NA, you need to tell the summarizing function to ignore them. Usually we add the argument `na.rm=TRUE`. For example, `mean(x, na.rm=TRUE)`.

4.4 Summarizing by groups in R

To summarize data by group in R, use the function `aggregate()`. For this example, we will use the built-in *iris* dataset, [which contains measurements taken from 150 flowers of 3 different species of *Iris*](#).

We can use `aggregate` to make a table with the mean and SD of the petal length of each species in *iris*.

```
# calculate mean of each species
ag.petal <- aggregate(Petal.Length~Species, data=iris, mean)

# calculate sd of each species
ag.petal$SD <- aggregate(Petal.Length~Species, data=iris, sd)$Petal.Length
ag.petal
```

	Species	Petal.Length	SD
1	setosa	1.462	0.1736640
2	versicolor	4.260	0.4699110
3	virginica	5.552	0.5518947

Some people prefer to use `tapply()` instead of a second call to `aggregate()`:

```
ag2 <- aggregate(Petal.Length~Species, data=iris, mean)
ag2$sd <- tapply(iris$Petal.Length, iris$Species, sd)
ag2
```

	Species	Petal.Length	sd
1	setosa	1.462	0.1736640
2	versicolor	4.260	0.4699110
3	virginica	5.552	0.5518947

You can also use `tapply()` to build a data frame of summary statistics from scratch, although `aggregate()` is usually safer because of how it handles missing groups.

R function `aggregate()` does what Pivot Tables do in Excel: summarize by groups. However, R has a huge advantage over Excel in that it can summarize by any function you can define, as opposed to the short list of functions available in Excel. For example, you could count the number of values ≥ 3 or between arbitrary values like 3.5 and 4.7. A function defined and used within a function as shown below is called an **anonymous function**.

```
# count number of values >=3 with an anonymous function
ag3 <- aggregate(Petal.Length~Species, data=iris,
                  function(x){length(which(x>=4))})
ag3
```

	Species	Petal.Length
1	setosa	0
2	versicolor	39
3	virginica	50

```
# count number of values between 3.5 and 4.7
ag4 <- aggregate(Petal.Length~Species, data=iris,
                  function(x){length(which(x>3.5 & x<4.7))})
ag4
```

	Species	Petal.Length
1	setosa	0
2	versicolor	34
3	virginica	1

Here is an example where we count the number of missing values and number of infinite values...two very useful functions for figuring out why your code doesn't work! These examples also show the use of anonymous functions in `tapply()` as well as in `aggregate()`.

```
tapply(iris$Petal.Length, iris$Species,
       function(x){length(which(is.na(x)))})
```

setosa	versicolor	virginica
0	0	0

```
aggregate(Petal.Length~Species, data=iris,
          function(z){any(is.nan(z))})
```

	Species	Petal.Length
1	setosa	FALSE
2	versicolor	FALSE
3	virginica	FALSE

Tip

which()-craft

In expressions like the one above, where we selected observations that met a certain condition (e.g., `which(x>4)`), the function `which()` is not strictly necessary. The expression

5 Tests comparing groups

In a test for difference in mean or location, we are interested in whether observations differ between groups. Most of the time we are testing whether the central tendency differs between groups. A test of this kind evaluates a prediction like, “a typical value in group A is different from a typical value in group B”.

Alternatively, we can test for a difference in **location**: the relative position of random values from 2 or more groups. This is a slightly different statement: “values in group A tend to be greater than (or less than) values in group B”. Put another way: “If we pick a random value from group A, it will be reliably greater (or less than) than a random value from group B”. Tests for location are performed on the **rank order** of the values, not the values themselves.

When comparing groups, the data are usually best shown using a **boxplot**, also known as a **box-and-whiskers** plot. The exact implementation varies between software packages, but boxplots usually show the median, interquartile range (approximately), extreme values, and possible outliers (if any) Run the command `?boxplot.stats` to see a description of how boxplots are calculated in R.

5.1 Comparing 2 groups: the *t*-test

The *t*-test compares the means of two groups. There are many kinds of *t*-tests, but the case of comparing two means is the most common. Technically, this is called a *“two-sided, two-sample *t*-test”* or sometimes a **Student’s *t*-test**, although the latter term is not accurate because modern software uses a different method of accounting for unequal variances between groups (the “Welch-Satterthwaite correction”). Most of the time you can just say *t*-test and people will know what you mean.

t-test example

Here is an example of running a *t*-test on two samples. If you try the code yourself with a different random number seed (`set.seed()`), or without a random number seed, your numbers will be slightly different because of the random number generation.

```
# make some data
set.seed(123) # ensures same numbers each time RNG is done
x1 <- rnorm(n=20, mean=10, sd=4)
x2 <- rnorm(n=20, mean=15, sd=4)

# run the test
t.test(x1, x2)
```

Welch Two Sample t-test

```
data:  x1 and x2
t = -3.6974, df = 37.082, p-value = 0.0007013
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.545528 -1.911424
sample estimates:
mean of x mean of y
 10.56650  14.79497
```

You can also use a *t*-test on data within a data frame using the formula interface. Notice that this code is essentially the same as the code for `boxplot()` and `aggregate()`.

```
# grab 2 of the species in iris
use.species <- c("versicolor", "virginica")
iris.2spp <- iris[which(iris$Species %in% use.species),]

# remove the now-missing level of "setosa"
```

```
# (this will matter for the plot in the next code block)
iris.2spp$Species <- droplevels(iris.2spp$Species)

# run the t-test
t.test(Petal.Length~Species, data=iris.2spp)
```

Welch Two Sample t-test

```
data: Petal.Length by Species
t = -12.604, df = 95.57, p-value < 2.2e-16
alternative hypothesis: true difference in means between group versicolor and group virginica
95 percent confidence interval:
 -1.49549 -1.08851
sample estimates:
mean in group versicolor  mean in group virginica
               4.260                5.552
```

Show the difference using a boxplot:

```
boxplot(Petal.Length~Species, data=iris.2spp,
        xlab="Species", ylab="Petal length (cm)")
```

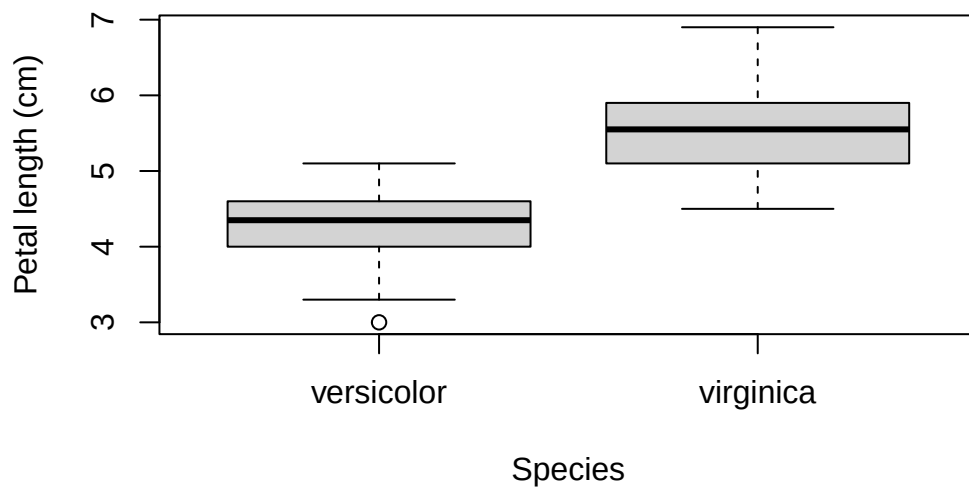


Figure 5: Boxplot showing that petal length is greater in *Iris virginica* than in *Iris versicolor*.

5.1.1 Non-parametric alternative

Sometimes, your data will not meet all of the assumptions of a *t*-test. That is beyond the scope of what we are doing here, but if you find yourself in that situation, you can try a **Wilcoxon rank sum test** instead of *t*-test. Their syntax is the same as for `t.test()`.

```
wilcox.test(Petal.Length~Species, data=iris.2spp)
```

Wilcoxon rank sum test with continuity correction

data: Petal.Length by Species

W = 44.5, p-value < 2.2e-16

alternative hypothesis: true location shift is not equal to 0

5.2 Comparing 3 or more groups: ANOVA

5.2.1 One-way ANOVA

When there are 3 or more groups, we do not use a t -test and instead use **analysis of variance**, or **ANOVA**. There are two ways to do an ANOVA in R, one using `aov()` and the other using `lm()`. We'll use `aov()`

When there is only 1 explanatory **factor** (grouping variable), regardless of how many **levels** (groups) it has, the test is called a **one-way ANOVA**.

```
# aov style
mod1 <- aov(Petal.Length~Species, data=iris)
summary(mod1)
```

```
              Df Sum Sq Mean Sq F value Pr(>F)
Species        2  437.1   218.55    1180 <2e-16 ***
Residuals     147    27.2     0.19
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The `summary()` command produces the **ANOVA table**, which summarizes how the variance is partitioned between different sources. In this case, the total sum of squared deviations of all observations is $437.1 + 27.2 = 464.3$. ANOVA partitions that into variation due to species (437.1) and residual or “left over” variation (27.2). We sometimes think of the 437.1 as “between group” variation, and the 27.2 as “within group” variation. If there is a significant effect of species, then roughly speaking there should be more between group variation than within group variation. Really it is the variation per degree of freedom (**Df**) that defines that ratio, or F statistic. Because of the way the table rounds values, it obscures the fact that $F = MS_{\text{between}} / MS_{\text{residuals}}$, or the ratio of the mean between group sums of squares and the mean within group sums of squares. The values shown, 281.55 and 0.19, are rounded so much that their quotient doesn't equal the printed F value.

Tip

Rounding errors

R is somewhat idiosyncratic in choosing how many digits it prints, and this can lead to confusion if values are rounded to too few decimal places. In the ANOVA example above, the MSE for the two sources of variation should divide to the F statistic. However, $218.55 / 0.19 \neq 1180$. If you look at the raw output of `mod1`, however, you that the sums of squares for Species and Residuals are 437.1028 and 27.2226, respectively. With more decimal places, you can then recalculate $(437.1028 / 2) / (27.2226 / 147) = 1180.161$, the

correct F ratio.

The **omnibus test** above shows that at least 1 level, or group, of **Species** differs from one of the others. To see the tests for the pairwise differences, we have to run a **post-hoc test**. One of the most common post-hoc tests for ANOVA is Tukey's Honest Significant Difference (HSD) test:

```
TukeyHSD(mod1)
```

```
Tukey multiple comparisons of means
 95% family-wise confidence level
```

```
Fit: aov(formula = Petal.Length ~ Species, data = iris)
```

```
$Species
```

	diff	lwr	upr	p adj
versicolor-setosa	2.798	2.59422	3.00178	0
virginica-setosa	4.090	3.88622	4.29378	0
virginica-versicolor	1.292	1.08822	1.49578	0

The Tukey output shows us that every group differs from every other, with $p < 0.001$ (here p is rounded to 0, although it can never be 0). The differences in group means are presented with their **95% confidence intervals (CI)**. For example, the difference between the group means of versicolor and setosa is 2.798 cm, with 95% CI = [2.594, 3.002]. Notice that none of these CI include 0: this suggests that a true difference of 0 between any two groups is very unlikely. Even without examining p -values, CI provide a useful way to evaluate both the direction and magnitude of differences.

One of the nice things about the Tukey HSD test is that the p -values are automatically adjusted for **multiple comparisons**, a phenomenon where the probability of a false positive goes up as a [researcher performs more tests](#), using a customized version of the probability distribution for the test statistic. This controls the **family wise error rate**, or total false positive probability for this set of three tests.

5.2.2 Two-way ANOVA (and more ways)

When there are 2 or more explanatory factors, the test is called a **two-way ANOVA**, or **three-way ANOVA**, and so on. We can add additional factors to an ANOVA with the + sign.

```
# make a copy of iris with an extra grouping variable
iris2 <- iris
iris2$Flower <- c("purple", "pink")

# fit the two-way ANOVA without interaction
mod2 <- aov(Petal.Length~Species+Flower, data=iris2)
summary(mod2)
```

```

              Df Sum Sq Mean Sq  F value Pr(>F)
Species        2  437.1   218.55  1174.229 <2e-16 ***
Flower          1    0.0     0.05    0.261   0.61
Residuals     146   27.2     0.19
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Adding the colors that way just repeated them until the column was full: purple, pink, purple, pink, and so on until the new vector had length 150 (the number of rows in `iris`). This essentially distributes the colors randomly with respect to the species and thus the petal lengths, so it should be no surprise that flower color had no effect on petal length ($p = 0.61$). A Tukey test confirms the effects of species, and the non-effect of `Flower`. Technically, you don't need to do or report a Tukey test if the main effect is nonsignificant.

```
TukeyHSD(mod2, which="Species")
```

```

Tukey multiple comparisons of means
 95% family-wise confidence level
```

```
Fit: aov(formula = Petal.Length ~ Species + Flower, data = iris2)
```

```
$Species
              diff      lwr      upr p adj
versicolor-setosa  2.798 2.593692 3.002308    0
virginica-setosa   4.090 3.885692 4.294308    0
virginica-versicolor 1.292 1.087692 1.496308    0
```

Something we could also ask is whether there was an **interaction** between species and color: a situation where one variable changes the effect of another. Maybe purple and pink flowers really do have different petal lengths, but only in one species. Testing for an interaction would help detect that kind of pattern. To test for an interaction between two variables, use the `*` symbol instead of the `+`.

```
# fit the two-way ANOVA WITH interaction
mod3 <- aov(Petal.Length~Species*Flower, data=iris2)
summary(mod3)
```

```

              Df Sum Sq Mean Sq  F value Pr(>F)
Species        2  437.1   218.55 1161.375 <2e-16 ***
Flower         1    0.0     0.05   0.258  0.612
Species:Flower  2    0.1     0.04   0.201  0.818
Residuals     144   27.1     0.19
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The interaction is nonsignificant, so we should instead report the main effects in the model without the interaction (mod2). If there is an interaction present, then it becomes impossible to discuss the effects of one variable without discussing the effects of the other. We will explore interactions more in the section on ANCOVA below.

5.3 Nonparametric alternatives

If your data do not meet the assumptions for ANOVA, then you can use a **Kruskal-Wallis test**. This is sometimes called a “nonparametric ANOVA” but that isn’t quite accurate. Essentially, the Kruskal-Wallis test compares the relative rank order of observations between groups to test whether values in each group tend to be consistently larger than or smaller than other groups.

```
# run the Kruskal-Wallis test
mod4 <- kruskal.test(Petal.Length~Species, data=iris)
mod4
```

```
Kruskal-Wallis rank sum test
```

```
data:  Petal.Length by Species
Kruskal-Wallis chi-squared = 130.41, df = 2, p-value < 2.2e-16
```

Similar to what happened in the one-way ANOVA example above, the omnibus test was significant, so we need to run a post-hoc test. The most common post-hoc test for a Kruskal-Wallis test is the **Dunn test**, available in package `dunn.test`.

```
# install the package if needed
install.packages("dunn.test")

# load the package so you can use its functions
library(dunn.test)
```

Now, we can perform the Dunn's test to see which groups likely differ from each other, much like we used the Tukey HSD test above.

```
dunn.test(iris$Petal.Length, g=iris$Species, method="holm")
```

Kruskal-Wallis rank sum test

data: x and group

Kruskal-Wallis chi-squared = 130.411, df = 2, p-value = 0

Comparison of x by group
(Holm)

Col Mean-		
Row Mean	setosa	versicol
-----+-----		
versicol	-5.862996	
	0.0000*	
virginic	-11.41838	-5.555388
	0.0000*	0.0000*

alpha = 0.05

Reject Ho if p <= alpha/2

The output shows, for each pair of factor levels, the differences in median ranks between groups (the top number), and the adjusted p -value for each comparison. The p -value is adjusted for multiple comparisons. In this case, we used a [Holm correction](#).

6 Correlation

Correlation is a statistical relationship where changes in one variable are reliably associated with changes in another variable. This relationship need not be causal, hence the famous mantra, “correlation does not equal causation”. As such, correlation is usually a **descriptive technique**,

which lets researchers characterize patterns, rather than an **inferential technique**, which lets researchers make estimates about unknown parameters (i.e., infer unobserved quantities).

Correlations are typically expressed as a **correlation coefficient**, a value that indicates the strength of the correlation by its magnitude and the direction of the correlation by its sign.

- **Positive correlation:** as one variable gets larger, the other variable does too. Correlation coefficient approaches $+1$.
- **Negative correlation:** as one variable gets larger, the other variable gets smaller. Correlation coefficient approaches -1 .

When a correlation coefficient is near 0, this indicates no correlation.

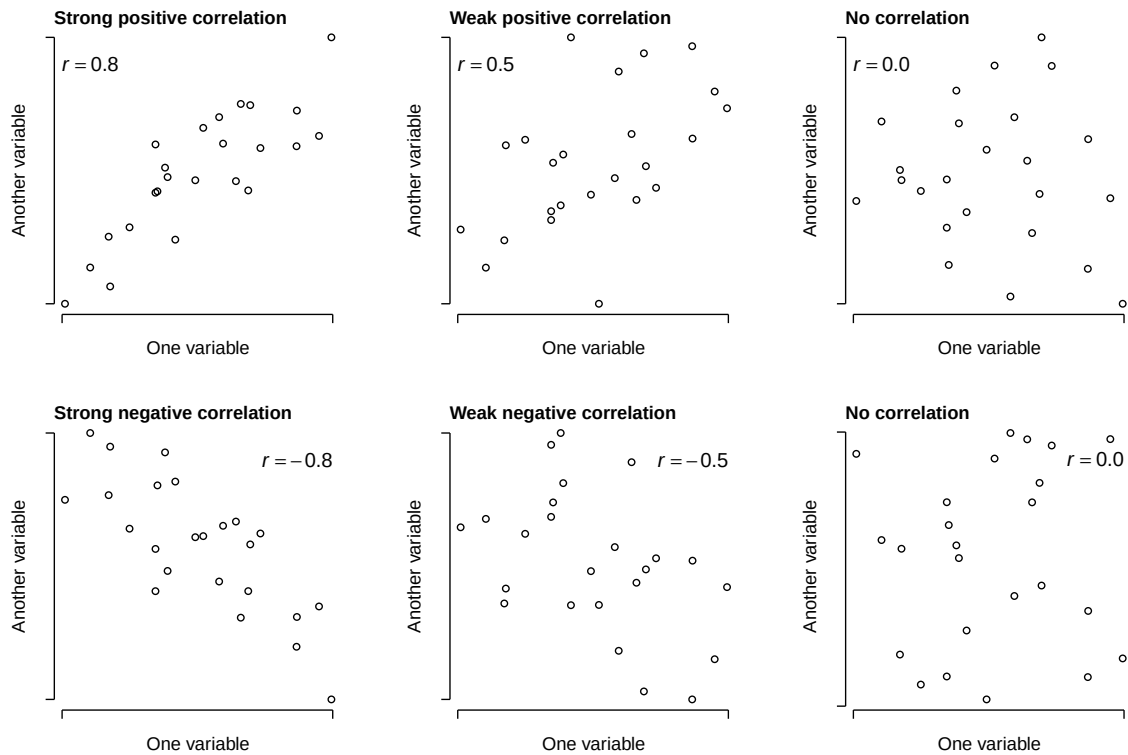


Figure 6: Illustration of possible positive and negative relationships between pairs of variables, with different levels of consistency (r).

6.1 Linear correlation

When we suspect that two variables are correlated, and that the shape of that relationship is a straight line, we can use **Pearson's product moment correlation**, also called **Pearson's**

r.

```
# calculate linear correlation coefficient
cor(iris$Petal.Length, iris$Petal.Width)
```

```
[1] 0.9628654
```

You can also request a significance test on the coefficient, although some might consider this unnecessary because correlation is inherently a descriptive technique. If you are using correlation as a hypothesis test, however, you do need a *p*-value.

```
cor.test(iris$Petal.Length, iris$Petal.Width)
```

Pearson's product-moment correlation

```
data: iris$Petal.Length and iris$Petal.Width
t = 43.387, df = 148, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.9490525 0.9729853
sample estimates:
      cor
0.9628654
```

One of the neat features of `cor()` is that you can get a correlation matrix showing the correlation coefficients between many pairs of variable at once.

```
# correlation matrix between the numerical variables in iris
cor(iris[,1:4])
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
Sepal.Length	1.0000000	-0.1175698	0.8717538	0.8179411
Sepal.Width	-0.1175698	1.0000000	-0.4284401	-0.3661259
Petal.Length	0.8717538	-0.4284401	1.0000000	0.9628654
Petal.Width	0.8179411	-0.3661259	0.9628654	1.0000000

Here we see that each variable is perfectly correlated with itself (the 1s on the diagonal). Sepal length and sepal width are negatively correlated (-0.1175698) while petal length and petal

width are strongly positively correlated (0.8717538). You only need to report correlation coefficients to 2 or 3 decimal places.

There isn't a built-in way to check for significant correlations between many pairs of variables at once. This is because `cor.test()` only works with one pair at a time. Fortunately, it's not hard to do in a loop:

```
# data frame with just the numeric variables
dx <- iris[,1:4]

# set up data frame to hold results
# combn(..., 2) returns matrix of all unique pairs of names
# t(ranspose) this and turn into a data frame.
cor.df <- as.data.frame(t(combn(names(dx), 2)))
cor.df$r <- NA
cor.df$p <- NA

# find the r and p values in a loop
for(i in 1:nrow(cor.df)){
  # test once per iteration
  curr <- cor.test(dx[,cor.df$V1[i]], dx[,cor.df$V2[i]])
  # pull out values we need from correlation test result
  # and round to 3 decimal places
  cor.df$r[i] <- round(curr$estimate,3)
  cor.df$p[i] <- round(curr$p.value, 3)
}#close i loop

# print result
cor.df
```

	V1	V2	r	p
1	Sepal.Length	Sepal.Width	-0.118	0.152
2	Sepal.Length	Petal.Length	0.872	0.000
3	Sepal.Length	Petal.Width	0.818	0.000
4	Sepal.Width	Petal.Length	-0.428	0.000
5	Sepal.Width	Petal.Width	-0.366	0.000
6	Petal.Length	Petal.Width	0.963	0.000

If you want to visualize many potential correlations at once, you can construct a **scatterplot matrix** using function `pairs()`. This will plot every pair of variables in a data frame. We are going borrow a function called `panel.cor()` from the `pairs()` help page so that we get both the scatterplots *and* the linear correlation coefficients.

```

# from ?pairs
# just paste this in once to define the function
# try changing some arguments to change the behavior!
panel.cor <- function(x, y, digits = 2, prefix = "", cex.cor, ...)
{
  par(usr = c(0, 1, 0, 1))
  r <- abs(cor(x, y))
  txt <- format(c(r, 0.123456789), digits = digits)[1]
  txt <- paste0(prefix, txt)
  if(missing(cex.cor)) cex.cor <- 0.8/strwidth(txt)
  text(0.5, 0.5, txt, cex = cex.cor * r)
}

# make the scatterplot matrix
pairs(iris[,1:4], lower.panel=panel.smooth, upper.panel=panel.cor, gap=0)

```

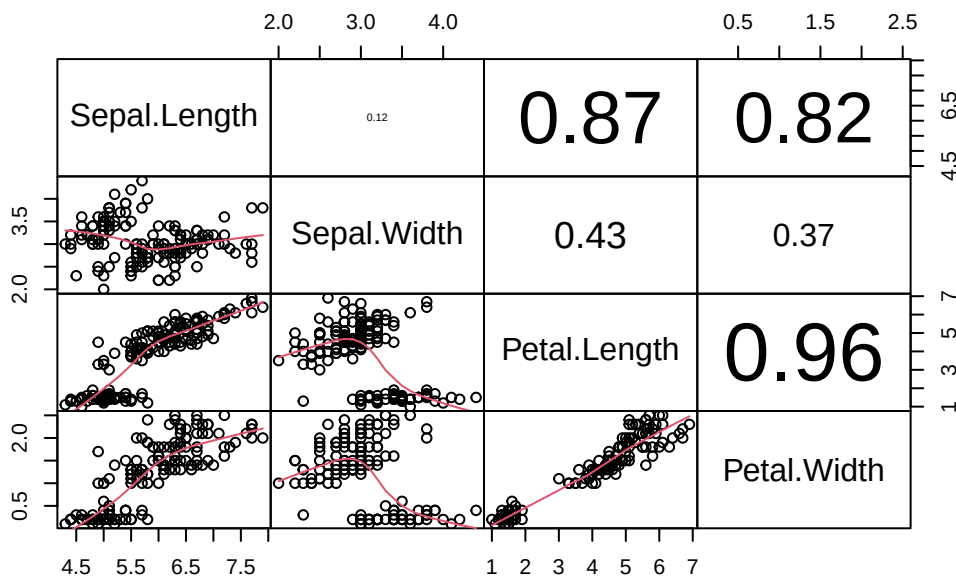


Figure 7: Scatterplot matrix showing possible correlations between the numeric variables in the iris data frame.

In this figure, scatterplots in the lower triangle correspond to the variables in the diagonal whom they intersect. For example, the far left and far bottom scatterplot has Sepal.Length for its horizontal axis, and Petal.Width for its vertical axis. Go one panel to the right, and

the horizontal axis is now Sepal.Width. The correlation coefficients in the upper triangle are identified the same way. For example, the large 0.96 on the far right, 3rd panel down, is the correlation coefficient between Petal.Length and Petal.Width.

6.2 Nonlinear correlation

Many processes and relationships in biology are not linear, and sometimes we want to calculate correlation coefficients for those relationships. Consider the figure below. The left panel clearly shows some relationship between X and Y , but it is not linear and using a linear correlation coefficient might miss something.

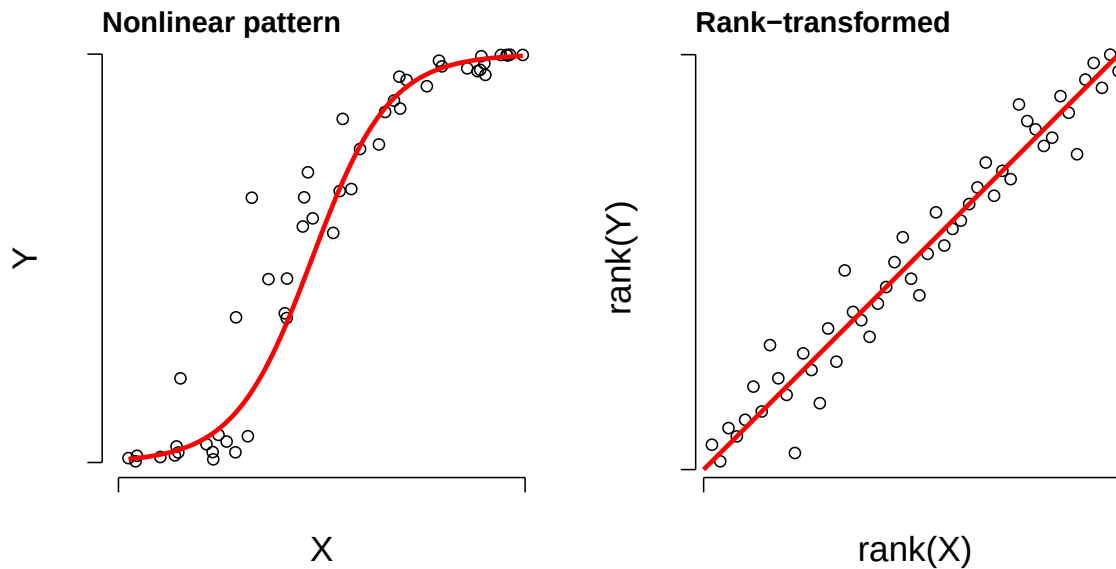


Figure 8: Left: illustration of a nonlinear correlation. Right: When values are rank-transformed, the nonlinear trend becomes linear.

However, if you **rank-transform** the data, the relationship turns into something more linear. A “rank-transform” assigns the smallest value value 0, the second smallest value to 1, and so on. Rank-transforming these data lets us calculate a nonparametric, nonlinear correlation. The nonparametric equivalent of Pearson’s r is **Spearman’s ρ** (“rho”). It can be obtained using code similar to that for linear correlation.

```
# just the correlation
cor(iris$Petal.Length, iris$Sepal.Width, method="spearman")
```

```
[1] -0.3096351
```

```
# significance test
cor.test(iris$Petal.Length, iris$Sepal.Width, method="spearman")
```

```
Warning in cor.test.default(iris$Petal.Length, iris$Sepal.Width, method =
"spearman"): Cannot compute exact p-value with ties
```

Spearman's rank correlation rho

```
data: iris$Petal.Length and iris$Sepal.Width
S = 736637, p-value = 0.0001154
alternative hypothesis: true rho is not equal to 0
sample estimates:
      rho
-0.3096351
```

When you run a nonlinear correlation test, you will often get the warning `Warning in cor.test.default ... : Cannot compute exact p-value with ties`. All this means is that some of the rankings are tied. This warning can be safely ignored.

7 Linear models

Linear models describe relationships where a change in some response variable Y is described by a unit change in a predictor X multiplied by a constant called the **slope**. In other words, if X increases by 1, then Y increases by the slope. This change in Y per unit change in X is the same for all X , so a graph of the model is a straight line. This is why linear models are *called* linear models.

7.1 Lots of analyses are linear models in disguise

Many models that people fit to data, including some that appear nonlinear, are actually linear models in disguise. This is because transforming one or both of X and Y can “straighten out” a nonlinear curve into a linear one.

7.1.1 Ordinary linear models

The basic linear regression model is written as

$$y = \beta_0 + \beta_1 x$$

and is the same as the equation for a line that you learned in algebra: $y = mx + b$. Statisticians just use β_0 for the Y -intercept and β_1 for the slope instead of b and m .

Why it's linear: Rate of change dy/dx is a scalar (β_1) and constant for all X .

7.1.2 Polynomials

A polynomial is a function such as $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \dots + \beta_k X^k$, where k is the **order** of the polynomial. A polynomial of order 2 is a quadratic equation. A polynomial of order 3 is a cubic, and so on. Polynomials have well-known shapes, such as the parabola of the quadratic function.

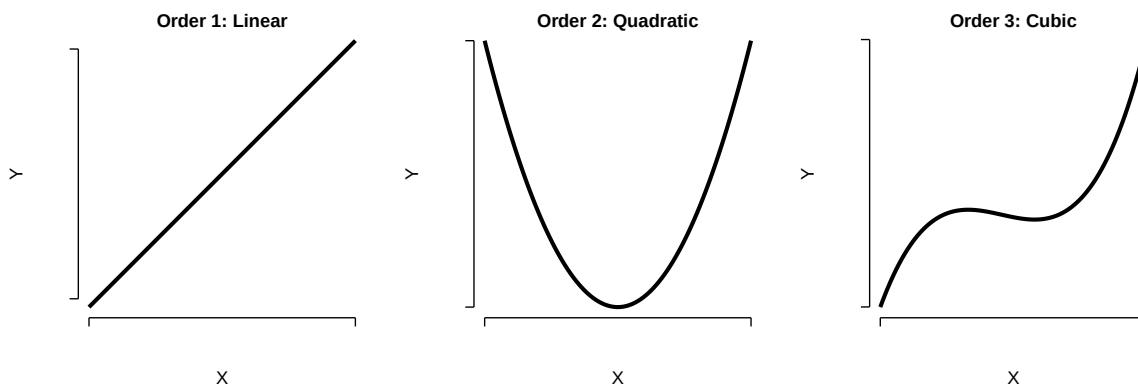


Figure 9: Curve shapes produced by polynomial equations of order 1, 2, and 3: linear, quadratic, and cubic, respectively.

Why it's linear: Although the rate of change dy/dx of a polynomial is a function of x and not a constant, the rate of change with respect to any x term *is* a constant. E.g., $dy/(dx^2) = \beta_2$. Compare the graph above to the graph below. By squaring or cubing the X term, we straightened out the line.

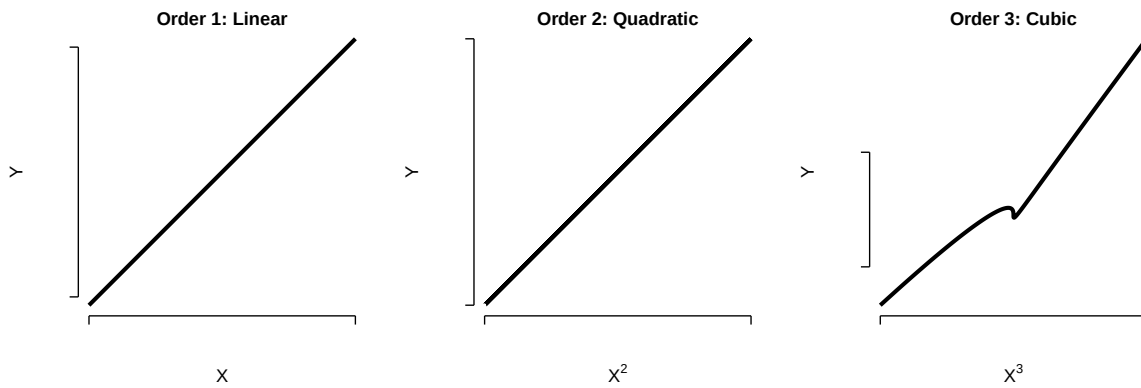


Figure 10: Polynomial functions become linear (straight) when plotted against X raised to the order of the polynomial. This is why polynomials can be treated as linear models.

7.1.3 Exponentials

The exponential function $y = ae^{bx}$ is ubiquitous in biology because it describes compounding growth (like compound interest).

Why it's linear: While the exponential function appears nonlinear, it can be made linear by log-transforming both sides:

$$\log(y) = \log(ae^{bx})$$

$$\log(y) = \log(a) + \log(e^{bx})$$

and because $\log(e^z) = z$:

$$\log(y) = \log(a) + bx$$

which is the form of a straight line with intercept $\log(a)$ and slope b .

💡 Tip

Logarithm bases

Mathematicians and statisticians almost always work with the **natural logarithm**, which uses Euler's number e (≈ 2.718282) as its base. The natural logarithm can be written $\log(x)$, $\log_e(x)$, or $\ln(x)$. Natural log is used because many [equations related to growth or change become easier](#) when e is the base. In this tutorial, and any math or statistics text, assume natural logarithm.

In contrast, scientists and engineers often use other bases, like $\log_{10}$ or $\log_2$, because these are more intuitive for humans. You can use whatever base you like, but be explicit about what you did.

7.1.4 Power laws

Power laws of the form $Y = aX^b$ are also ubiquitous in biology, as they describe proportional growth. Power laws are similar to exponential functions, but with a key difference: in a power law, X is in the base and is raised to a power, while in an exponential, X is in the exponent of some other base.

Why it's linear: Like an exponential, a power law can be linearized by taking the logarithm of both sides.

$$\log(Y) = \log(aX^b)$$

$$\log(Y) = \log(a) + b \times \log(X)$$

In this form, the power law is linear with intercept $\log(a)$ and slope b .

7.1.5 t -tests and ANOVA

This notion of linearity is what makes many analyses with factors (categories or grouping variables) as predictors “linear”. For example, the t -test is a linear model, but it doesn’t really describe a line on a plot of y vs. x the way that linear regression does. However, if you encode the factor (grouping variable) as 0s and 1s and think about the underlying algebra, it becomes clear what the difference in group means really means:

$$E(y_i) = (1 \times \beta_0) + (x_i \times \beta_1)$$

For group 1, $x_1 = 0$, so the expected value is β_0 . For group 2, $x_i = 1$, so the β_1 term comes into play as the difference in means between groups 1 and 2. This is what R, SAS, and other programs are doing internally when you analyze data with a factor variable. Here is what that looks like visually:

Similar logic explains why ANOVA with 3 or more levels is also a linear model.

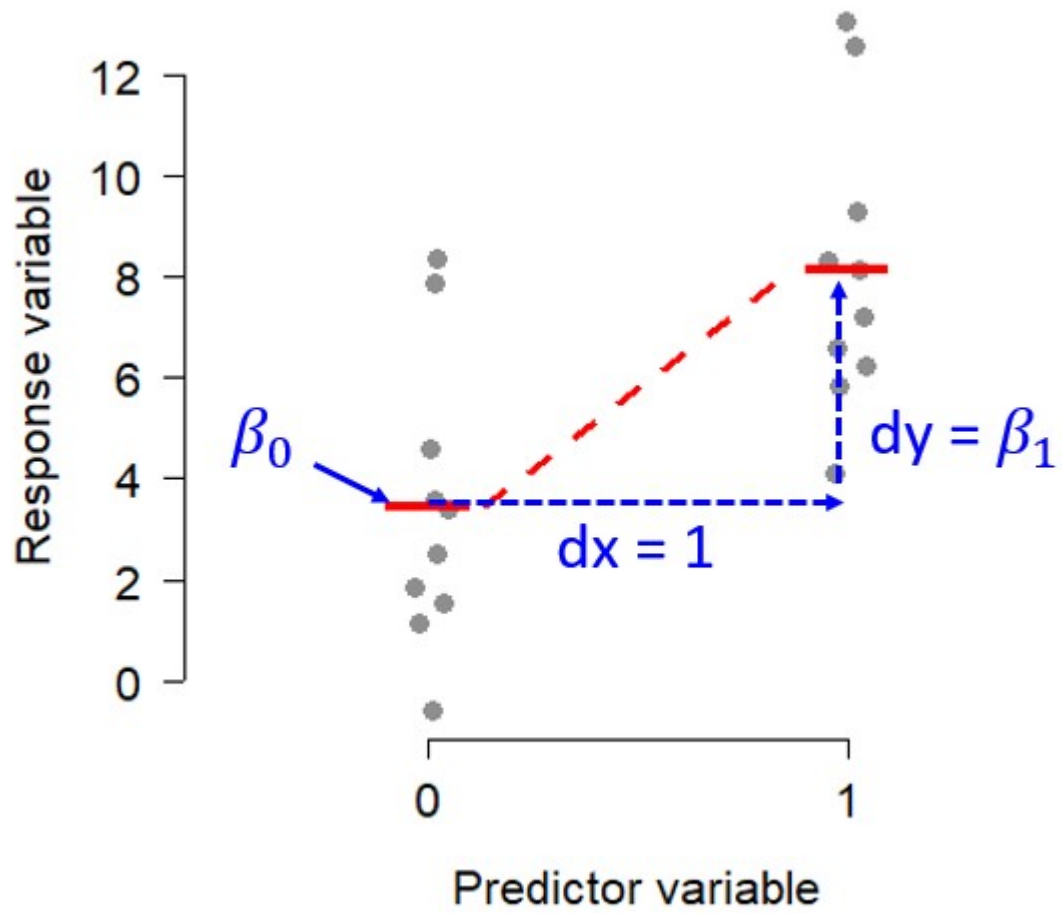


Figure 11: Illustration of why a t-test is a linear model.

7.2 Assumptions of linear models

As powerful and useful as linear models can be, they are not without assumptions that must be met. There are 4 assumptions that your data must meet before using linear models:

Normal residuals: The linear model assumes that model residuals, the random errors representing the differences between observed and predicted values, follow a normal distribution. Models for data that do not have normal residuals exist, but they are not linear models.

Homoscedasticity, or constancy of variance: Linear models assume **homoscedasticity**, or constant variance. This means that the residual variance does not depend on either X or Y . This is part of the definition of a variable or a residual being independent and identically distributed (i.i.d.). If the variance in the response variable depends on some predictor variable, then this should be incorporated into the model (resulting in something other than a linear model). If the variance appears to depend on the response variable, then there is probably an issue with the assumed response distribution and you should consider using GLMs (below) or mixed models (outside the scope of this tutorial).

Here is what homoscedasticity and heteroscedasticity (unequal variance) look like in a scatter-plot:

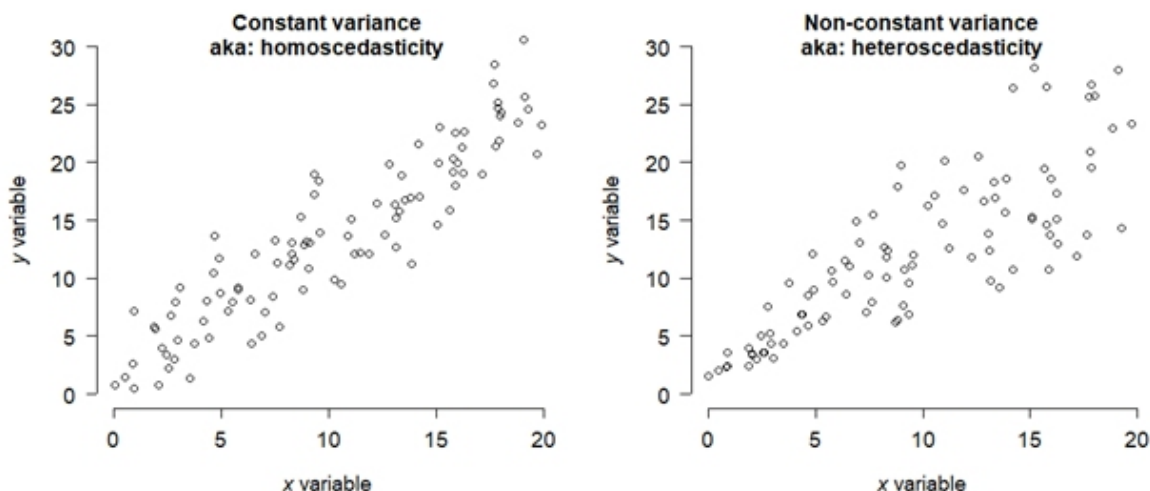


Figure 12: Illustration homoscedasticity and heteroscedasticity in a scatterplot.

Here is what they look like in a boxplot.

Heteroscedasticity is a serious problem for linear models because it leads to biased parameter estimates and standard errors (SE) of those estimates. The latter issue means that significance tests on parameters will be incorrect. Bayesian statistics does not involve significance tests, but heteroscedasticity that is unaccounted for will still lead to biased estimates. The usual solution

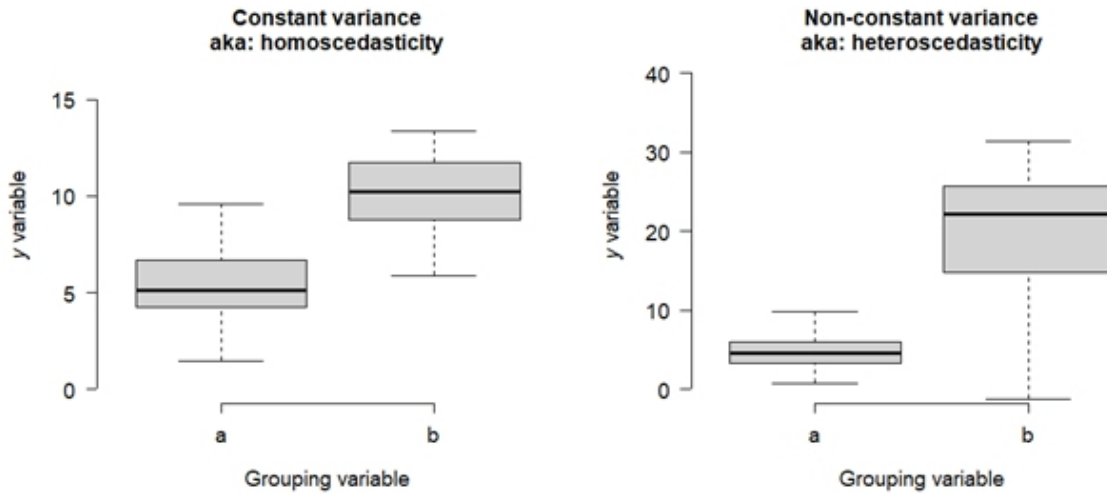


Figure 13: Illustration of homoscedasticity and heteroscedasticity in a boxplot.

is to either apply a variance-stabilizing transformation (like the log-transform), incorporate variance parameters that can be estimated, or be clever about choosing a response distribution.

Fixed and independent predictors: Linear models assume that predictor values are precisely known (“fixed”) and independent of each other (i.e., not autocorrelated). If there is uncertainty in the predictor variables, this adds uncertainty to the response values that linear models cannot account for. Simply put, linear models have a term for uncertainty in y , σ_{res} , but no term for uncertainty in X .

Independently and identically distributed errors: The assumption of independently and identically distributed (i.i.d.) errors is very important. It means that the residual, or predictive error, for each observation depends only on the parameters of the residual distribution and *not* on predictor variables or other observations. When the assumption of independence is violated, the degrees of freedom in the analysis is artificially inflated because the number of unique pieces of information is smaller than the nominal number of samples. In frequentist analyses, this in turn deflates the p -value and increases the chance of a type I error (false positive). There are methods to deal with errors that are not independent, but linear models are not among them.

Final note about assumptions: Linear models can be robust to *mild* violations of these assumptions, but you need to be careful. Many of these assumptions can be tested directly—for example, testing residuals for normality or homoscedasticity—and if there is any doubt about whether your data meet the assumptions, you should perform those tests. Actually, you should perform the tests anyway just to be safe.

7.3 Linear regression

The linear regression model describes a pattern where the response variable Y increases as a linear function of the predictor variable X . It looks like this:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

where β_0 is the y -intercept, β_1 is the slope (effect of a unit increase in X on Y), and ε is residual variation in Y not explained by X —the spread of points above or below the line. In a linear model, residual variation follows a normal distribution with mean 0 and some unknown SD σ_{res} . Each residual ε_i follows the same distribution:

$$\varepsilon_i \sim \mathcal{N}(0, \sigma_{\text{res}})$$

where $\mathcal{N}$ signifies the normal distribution.

The R syntax for linear regression is basically the same as that for ANOVA and the t -test above, but this time the right hand side of the formula has a continuous predictor rather than a factor.

```
# fit a linear regression model
mod5 <- lm(Petal.Length~Petal.Width, data=iris)
```

```
# test for significance of terms
anova(mod5)
```

Analysis of Variance Table

```
Response: Petal.Length
      Df Sum Sq Mean Sq F value    Pr(>F)
Petal.Width  1 430.48   430.48  1882.5 < 2.2e-16 ***
Residuals 148  33.84     0.23
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# see estimates of parameter coefficients
summary(mod5)
```

```

Call:
lm(formula = Petal.Length ~ Petal.Width, data = iris)

Residuals:
    Min       1Q   Median       3Q      Max
-1.33542 -0.30347 -0.02955  0.25776  1.39453

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.08356     0.07297   14.85  <2e-16 ***
Petal.Width  2.22994     0.05140   43.39  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4782 on 148 degrees of freedom
Multiple R-squared:  0.9271,    Adjusted R-squared:  0.9266
F-statistic: 1882 on 1 and 148 DF,  p-value: < 2.2e-16

```

The `summary()` output tells you a lot about the regression: the coefficients (and their p -values), the omnibus ANOVA for the model (the F test at the bottom), and the coefficient of determination (Adjusted R -squared). Coefficients are identified as the y-intercept β_0 (**Intercept**) and by the variable names (e.g., the slope with respect to petal width is **Petal.Width**). Each coefficient is presented as its estimate and SE (e.g., the intercept is 1.08 ± 0.07). The test statistic t for each coefficient is the ratio of the estimate to the SE, and the P -value for t calculated from a t -distribution (much as in a t -test). The coefficient of determination R^2 is the proportion of variation in Y explained by the model. In this case, about 93% of variation in **Petal.Length** is explained by **Petal.Width**.

When you present a linear regression, you should present the parameters, their SE, test statistics, and p -values in a table. The omnibus F test and R^2 can be reported in Results text. Finally, you should present the data as a scatterplot, with the predicted values as a trendline and the 95% CI of that trendline if feasible. Here is how to do that for the model above:

```

# generate x values over which to draw trendline
n <- 100
px <- seq(min(iris$Petal.Width), max(iris$Petal.Width), length=n)

# put in a data frame with same variable names as in the model
prx <- data.frame(Petal.Width=px)

# calculate predicted values and their SE
pred <- predict(mod5, newdata=prx, se.fit=TRUE)

```

```

# put predicted value and 95% confidence limits into data frame
# safest method is to draw quantiles from normal distribution
# defined by predicted value (pred$fit) and SEs (pred$se.fit)
prx$mn <- qnorm(0.5, mean=pred$fit, sd=pred$se.fit)
prx$lo <- qnorm(0.025, mean=pred$fit, sd=pred$se.fit)
prx$up <- qnorm(0.975, mean=pred$fit, sd=pred$se.fit)

# make the plot
## include some par options for a more professional figure
## see ?par for definitions
par(mfrow=c(1,1), mar=c(5.1, 5.1, 1.1, 1.1),
    lend=1, las=1, bty="n", cex.axis=1.3, cex.lab=1.3)
## make the base plot
plot(iris$Petal.Width, iris$Petal.Length, pch=16, cex=1.2,
     xlab="Petal width (cm)", ylab="Petal length (cm)",
     xlim=c(0, 2.5), ylim=c(0, 8))
## add trendline and 95% confidence limits as lines with points(...,type="l")
points(prx$Petal.Width, prx$lo, type="l", lwd=2, col="red", lty=2)
points(prx$Petal.Width, prx$up, type="l", lwd=2, col="red", lty=2)
points(prx$Petal.Width, prx$mn, type="l", lwd=2, col="red")

```

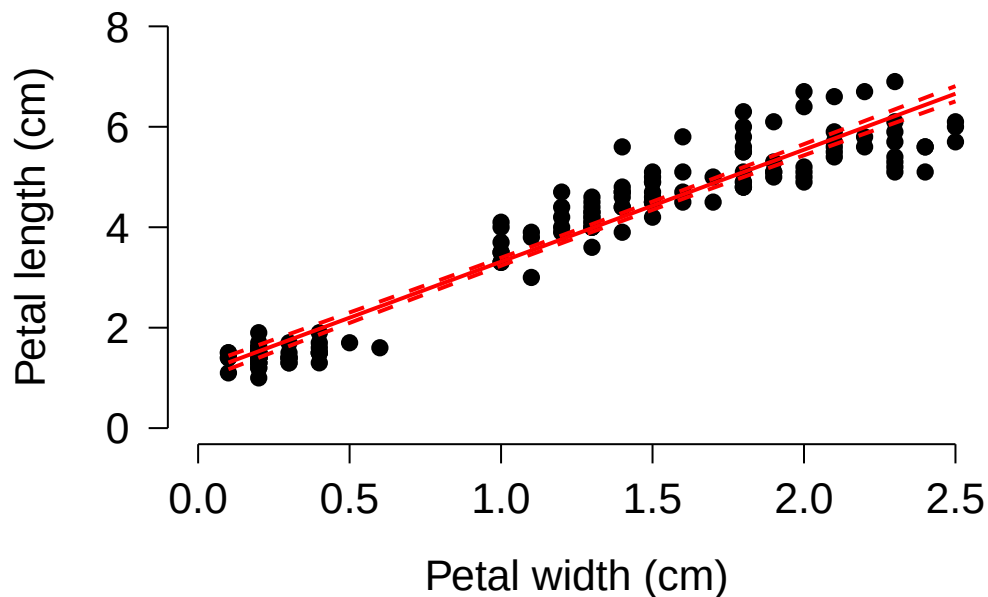


Figure 14: Data plotted with the trendline (solid) and its 95% confidence interval (dashed).

7.4 Multiple linear regression

Linear regression is easily extended to **multiple linear regression**, where there are >1 continuous predictors. Just as with ANOVA, additional predictor variables are added to the model formula using `+`.

```
mod6 <- lm(Petal.Length~Petal.Width + Sepal.Width, data=iris)
summary(mod6)
```

Call:

```
lm(formula = Petal.Length ~ Petal.Width + Sepal.Width, data = iris)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-1.33753	-0.29251	-0.00989	0.21447	1.24707

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	2.25816	0.31352	7.203	2.84e-11 ***
Petal.Width	2.15561	0.05283	40.804	< 2e-16 ***
Sepal.Width	-0.35503	0.09239	-3.843	0.00018 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4574 on 147 degrees of freedom

Multiple R-squared: 0.9338, Adjusted R-squared: 0.9329

F-statistic: 1036 on 2 and 147 DF, p-value: < 2.2e-16

Adding predictors to a multiple regression model will almost always increase the model fit (e.g., proportion of variance explained, R^2), but this does not mean you should keep adding predictors. Multiple regression models are extremely vulnerable to **overfitting**, where random noise (i.e., residual variation) is modeled as if it was part of the deterministic part of the model. Multiple regression models are also highly vulnerable to **collinearity**, where the predictors are correlated with each other. When collinear predictors are included in a model, it is impossible for the model to unambiguously partition the sums of squares associated with each predictor.

7.5 Analysis of covariance (ANCOVA)

Sometimes biologists want to test for effects of one variable while controlling for another. For example, the slope between Y and X might differ between groups. Or, groups might differ only after accounting for some continuous covariate. Here are some specific examples:

- Herbivorous mammals tend to have longer intestines than carnivorous mammals, but only after accounting for the effect of body mass on intestine length. The researcher wants to see how groups differ, but needs to control for a continuous variable (body length) first.
- Humerus length and femur length both increase with body mass in birds, but humerus length increases faster than femur length (i.e., their slopes are different). The researcher wants to test for a difference in a continuous relationship, but needs to control for a factor (which bone) first.

The method for both situations is called **analysis of covariance**, or **ANCOVA**. You can think of ANCOVA as combining linear regression and ANOVA. ANCOVA is especially useful for studying **interactions**, where one variable changes the effect of another.

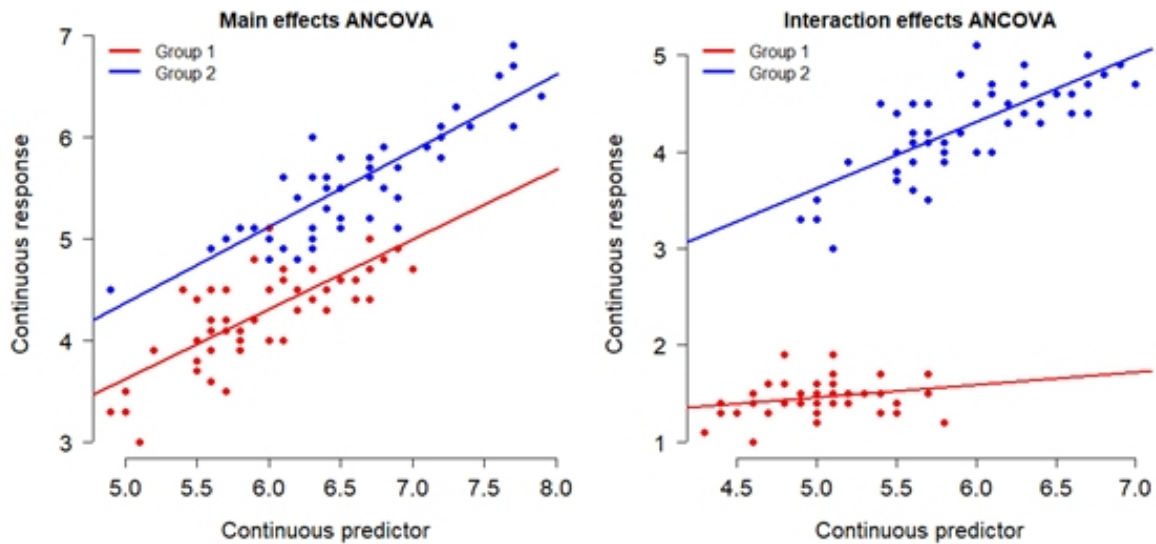


Figure 15: Illustration of ANCOVA without an interaction (different intercepts, same slope) and with an interaction (different intercepts and slopes.)

So that we really understand what an ANCOVA does, let's simulate our own dataset where groups A and B have different slopes.

```
# simulation parameters
set.seed(42)
n <- 30
beta0_a <- 2
beta0_b <- 6
beta1_a <- 1.4
beta1_b <- 2.5
sigma_res <- 2
```

```

# groups A and B start in separate data frames
data_a <- data.frame(group="A", col="red", pch=15, x=runif(n, 5, 20))
data_b <- data.frame(group="B", col="blue", pch=16, x=runif(n, 5, 20))

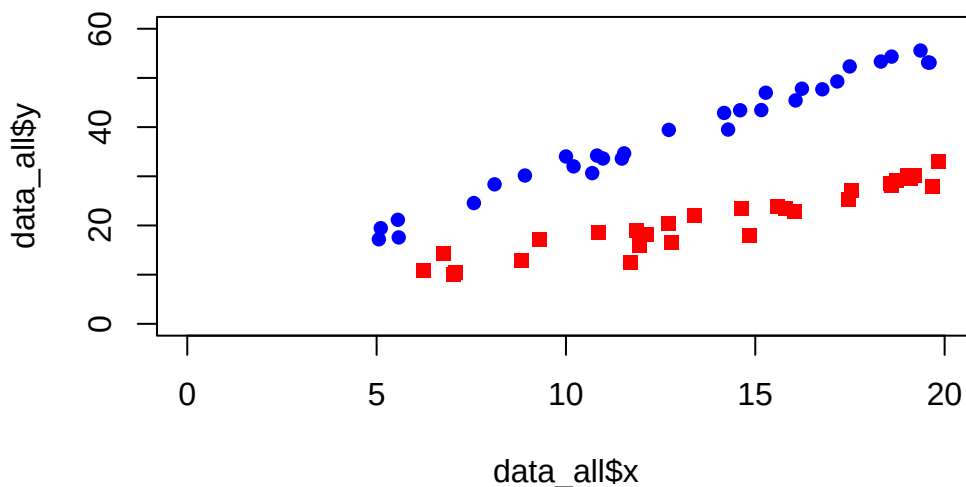
# add expected values
data_a$ey <- beta0_a + beta1_a * data_a$x
data_b$ey <- beta0_b + beta1_b * data_b$x

# combine to single data frame
data_all <- rbind(data_a, data_b)

# add residuals
data_all$y <- data_all$ey + rnorm(n*2, mean=0, sd=sigma_res)

# and make a plot to inspect our data
plot(data_all$x, data_all$y, col=data_all$col, pch=data_all$pch,
      xlim=c(0, 20), ylim=c(0, 60))

```



Now we can fit the ANCOVA and see the results.

```

# fit the model. Notice the * to specify an interaction
mod7 <- lm(y~x*group, data=data_all)

```

```
# test significance of terms
anova(mod7)
```

Analysis of Variance Table

Response: y

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
x	1	3507.2	3507.2	1012.289	< 2.2e-16 ***
group	1	5664.0	5664.0	1634.827	< 2.2e-16 ***
x:group	1	311.0	311.0	89.778	3.108e-13 ***
Residuals	56	194.0	3.5		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Now check the model parameters

```
summary(mod7)
```

Call:

```
lm(formula = y ~ x * group, data = data_all)
```

Residuals:

Min	1Q	Median	3Q	Max
-5.5122	-0.9491	0.4801	1.1691	3.8166

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.44990	1.17053	0.384	0.702170
x	1.49194	0.07883	18.925	< 2e-16 ***
groupB	5.62958	1.55677	3.616	0.000642 ***
x:groupB	1.03148	0.10886	9.475	3.11e-13 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.861 on 56 degrees of freedom

Multiple R-squared: 0.9799, Adjusted R-squared: 0.9789

F-statistic: 912.3 on 3 and 56 DF, p-value: < 2.2e-16

R treats group A as the control or **baseline** because it is alphabetically first. Any effect involving group B is relative to the effect in group A. Here:

- The intercept in group A is (Intercept), 0.4499.
- The slope in group A is x, 1.4919.
- The intercept in group B is the group A intercept plus the effect of being in group B: $0.4499 + 5.6295 = 6.0794$.
- The slope in group B is the slope in group A plus the effect of being in group B on the slope: $1.4919 + 1.0314 = 2.5233$

Making a plot with trendlines for an ANCOVA follows much the same procedure as we used to make the plot for linear regression, but, you have to take care to include each group.

```
# get x sequence within the x domain of each group
pxa <- seq(min(data_all$x[data_all$group=="A"]),
           max(data_all$x[data_all$group=="A"]),
           length=100)
pxb <- seq(min(data_all$x[data_all$group=="B"]),
           max(data_all$x[data_all$group=="B"]),
           length=100)
# put in data frames
prxa <- data.frame(group="A", x=pxa)
prxb <- data.frame(group="B", x=pxb)

# calculate predictions
preda <- predict(mod7, newdata=prxa, se.fit=TRUE)
predb <- predict(mod7, newdata=prxb, se.fit=TRUE)

# add to data frames
prxa$mn <- qnorm(0.5, preda$fit, preda$se.fit)
prxa$lo <- qnorm(0.025, preda$fit, preda$se.fit)
prxa$up <- qnorm(0.975, preda$fit, preda$se.fit)

prxb$mn <- qnorm(0.5, predb$fit, predb$se.fit)
prxb$lo <- qnorm(0.025, predb$fit, predb$se.fit)
prxb$up <- qnorm(0.975, predb$fit, predb$se.fit)

# make the plot
par(mar=c(5.1, 5.1, 1.1, 1.1), bty="n", las=1, lend=1,
    cex.axis=1.3, cex.lab=1.3)
plot(data_all$x, data_all$y, col=data_all$col, pch=data_all$pch,
     xlim=c(0, 20), ylim=c(0, 60))
points(prxa$x, prxa$mn, type="l", lwd=2, col="red")
points(prxa$x, prxa$lo, type="l", lwd=2, col="red", lty=2)
```

```

points(prxa$x, prxa$up, type="l", lwd=2, col="red", lty=2)
points(prxb$x, prxb$mn, type="l", lwd=2, col="blue")
points(prxb$x, prxb$lo, type="l", lwd=2, col="blue", lty=2)
points(prxb$x, prxb$up, type="l", lwd=2, col="blue", lty=2)
legend("topleft", legend=c("Group A", "Group B"),
      pch=c(15, 16), col=c("red", "blue"), cex=1.4,
      bty="n")

```

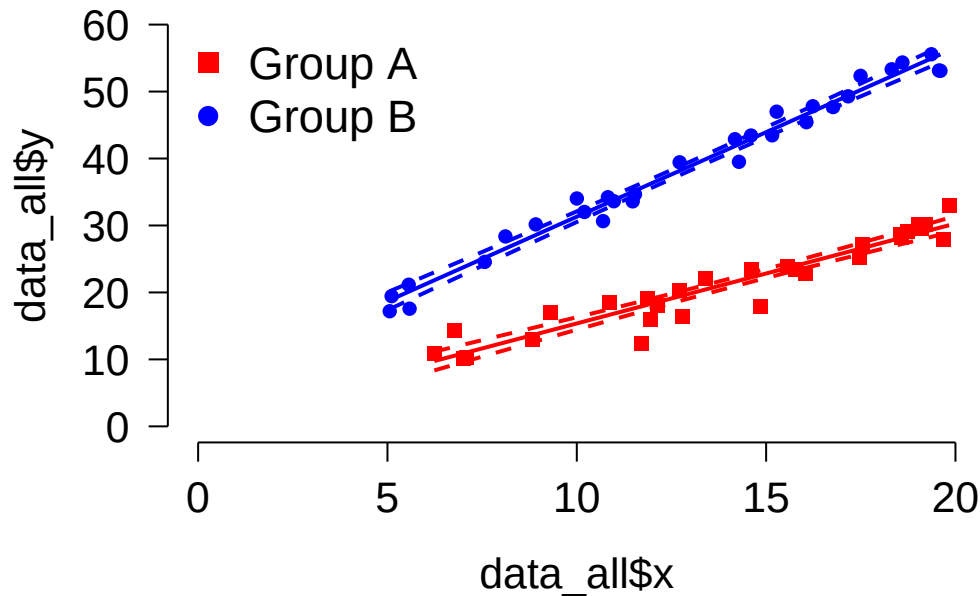


Figure 16: Plot of Y vs. X with groups A and B in the data, where the groups have different slopes. This illustrates ANCOVA with an interaction between X and group.

8 Generalized linear models

The **generalized linear model (GLM)** is a framework of related methods that generalize the linear model by relaxing two of the conditions in which linear models are used. In a GLM:

- The response variable can be modeled on a different scale than its original scale. The original scale of the data is related to the scale of the linear prediction by a **link function**.
- The values of the response variable can come from many different distributions, not just the normal distribution.

There are four GLMs that biologists commonly encounter. The first is linear regression, which we have already covered. Linear regression can be expressed as a GLM like this:

$$Y \sim \mathcal{N}(\mu, \sigma)$$
$$\mu = \eta$$

$$\eta = \beta_0 + \beta_1 X$$

The linear predictor, η (“eta”) is a linear function of X . The expected value of Y , μ , is equal to η . That is, μ is related to η by the identity function. Finally, observed values in Y are drawn from a normal distribution with mean μ and SD σ .

8.1 Models for counts: Poisson GLM

Another common GLM biologists encounter is the **Poisson GLM for count data**. Count data are exactly what they sound like: values that are counted and can only take on non-negative integer values. Because counts can only be non-negative integers, the normal distribution is not appropriate because it is supported on the entire real number line. Here is the Poisson GLM:

$$Y \sim \text{Poisson}(\lambda)$$

$$\log(\lambda) = \eta$$

$$\eta = \beta_0 + \beta_1 X$$

As with linear regression, the linear predictor η is a linear function of X . However, η is a logarithm of the expected value of Y , λ . So, the link function that relates the expected value λ to the linear predictor scale expected value η is the *log* function. Finally, observed Y values are drawn from a Poisson distribution with expected value (mean) λ . The single parameter λ is also the variance (σ^2) of the distribution.

Let’s simulate some data and fit a Poisson GLM.

```

# simulate some data
set.seed(321)
n <- 60
dx <- data.frame(x=runif(n, 0, 20))
beta0 <- 1.3
beta1 <- 0.65
dx$eta <- beta0 + beta1*dx$x
dx$y <- rpois(n, dx$eta)

# check the data
plot(dx$x, dx$y, ylim=c(0, 25))

```

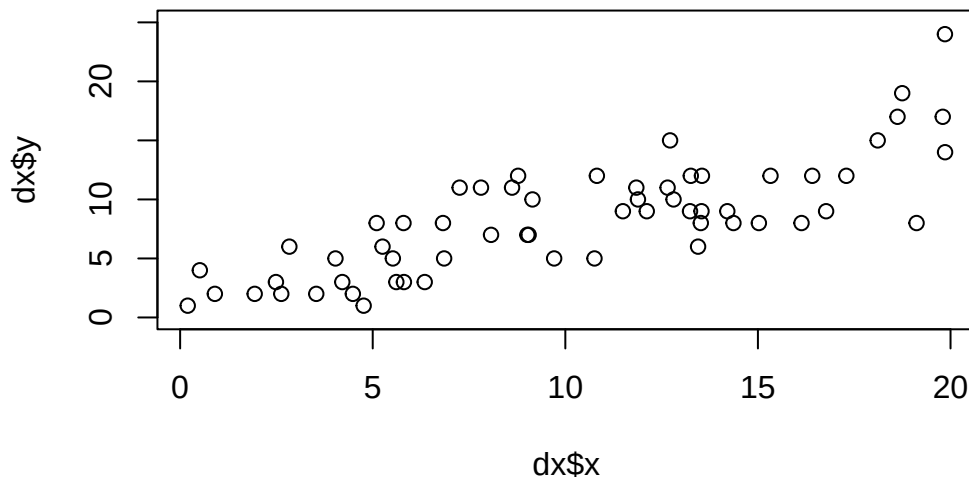


Figure 17: Plot of simulated count data, where Y increases with X .

Now fit the Poisson GLM. The omnibus test for most GLMs that aren't also linear models (e.g., linear regression or ANCOVA) is a likelihood ratio test against a **null model**—a model with only the mean as a predictor.

```

# fit the model
mod8 <- glm(y~x, data=dx, family=poisson(link="log"))

# fit a null model with no predictors, for comparison

```

```
mod8.null <- glm(y~1, data=dx, family=poisson(link="log"))

# Omnibus test: likelihood ratio test vs. null model
anova(mod8.null, mod8, test="Chisq")
```

Analysis of Deviance Table

```
Model 1: y ~ 1
Model 2: y ~ x
  Resid. Df Resid. Dev Df Deviance  Pr(>Chi)
1         59    161.939
2         58     61.671  1    100.27 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

This would be reported as “a likelihood ratio test found the model significantly different than the null model ($\chi^2_1 = 100.27$, $p < 0.001$.)

As usual, we can see the model parameters with `summary()`.

```
summary(mod8)
```

```
Call:
glm(formula = y ~ x, family = poisson(link = "log"), data = dx)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  1.180590    0.115666  10.207  <2e-16 ***
x             0.082894    0.008463   9.794  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 161.939  on 59  degrees of freedom
Residual deviance:  61.671  on 58  degrees of freedom
AIC: 293.45

Number of Fisher Scoring iterations: 4
```

We can get a good sense of how well the model fits the data by calculating a pseudo- R^2 as $(Dev_{null} - Dev_{res}) / Dev_{null}$, or in this case $(161.939 - 61.671) / 161.939 = 0.619$. The AIC, 293.45, doesn't mean much on its own but is useful for comparing models; generally, models with smaller AIC are better fit to the data.

You can then generate and draw a trendline much as we have before, with one wrinkle: calculate the expected values on the link scale, then exponentiate to get the values on the data scale. If you use `qpois()` to calculate expected values, you will get a step function rather than a smooth curve, because `qpois()` returns values from the Poisson distribution...which must be non-negative integers.

```
# x values
px <- seq(min(dx$x), max(dx$x), length=100)
# data frame
prx <- data.frame(x=px)
# predictions the correct way
pred.correct <- predict(mod8, newdata=prx, se.fit=TRUE, type="link")
prx$mn.correct <- exp(qnorm(0.5, pred.correct$fit, pred.correct$se.fit))
prx$mn.wrong1 <- qpois(0.5, exp(pred.correct$fit))

# predictions the wrong way
# (not plotted)
pred.wrong <- predict(mod8, newdata=prx, type="response")
prx$mn.wrong2 <- qpois(0.5, pred.wrong)

par(mar=c(5.1, 5.1, 1.1, 1.1), bty="n")
plot(dx$x, dx$y, pch=16, cex=1.5, ylim=c(0, 25))
points(prx$x, prx$mn.correct, type="l", lwd=3, col="red")
points(prx$x, prx$mn.wrong1, type="l", lwd=3, col="blue")
legend("topleft", legend=c("Correct", "Wrong"),
      lwd=3, col=c("red", "blue"), bty="n")
```

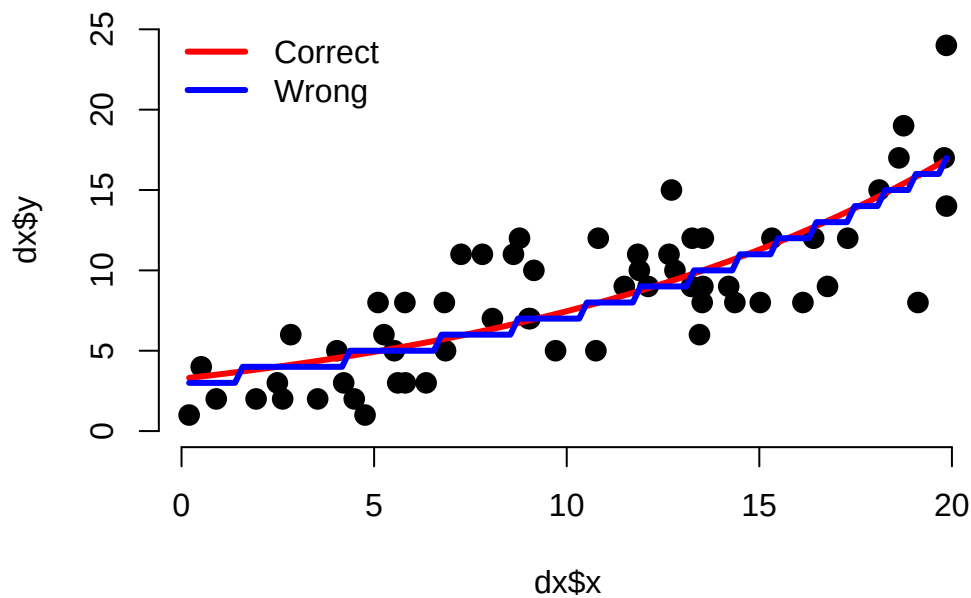


Figure 18: Plot of simulated count data, where Y increases with X , with a correctly generated trendline (red, smooth) and an incorrect trendline (blue, stepwise).

 Tip

Overdispersion

Sometimes count data can be **overdispersed**, in which case they have more variance than would be expected under a Poisson distribution. In such a case, you can try fitting a **negative binomial GLM**, which has an extra term to account for the excess variance. Negative binomial GLMs are available in package MASS.

8.2 Models for presence: Logistic Regression

Sometimes biologists need to model **binary outcomes**: response variables that can take on only one of two values such as alive or dead, presence or absent, or reproduced or did not reproduce. The tool for such variables is **logistic regression**.

The response variable Y takes on value 1 when some event occurs, and 0 otherwise. For example, if you are studying the presence of a rare mushroom species across many forest patches, you would code the sites with the mushroom 1 and the sites without the mushroom 0. Your analysis would then include a term for the probability of $Y = 1$, $p(Y = 1)$. Because probabilities can only occur in the closed interval $[0, 1]$, we cannot use linear models for them because—as with

the count data above—the normal distribution part of the linear model can produce nonsensical values.

Logistic regression is defined in observation-wise notation as (subscript i is for observation):

$$Y_i \sim \text{Bernoulli}(p_i)$$

$$\log\left(\frac{p_i}{1-p_i}\right) = \eta_i$$

$$\eta_i = \beta_0 + \beta_1 X_i$$

The link function $\log(x/(1-x))$ is the **logit function**, which maps probabilities in the open interval $(0, 1)$ to the real number line, the domain of the η . $\text{Logit}(0.5) = 0$; as p approaches 1, $\text{logit}(p)$ increases, and vice versa. This lets us use map a linear function to the bounded co-domain of probabilities.

As with the other GLMs, let's simulate our own dataset and fit the model to it.

```
set.seed(234)
n <- 60
dx <- data.frame(x=runif(n, 0, 12))
beta0 <- -4.3
beta1 <- 0.72
dx$eta <- beta0 + beta1*dx$x
dx$p <- plogis(dx$eta)
dx$y <- rbinom(n, 1, dx$p)

# check the data
plot(dx$x, dx$y, ylim=c(0, 1))
```

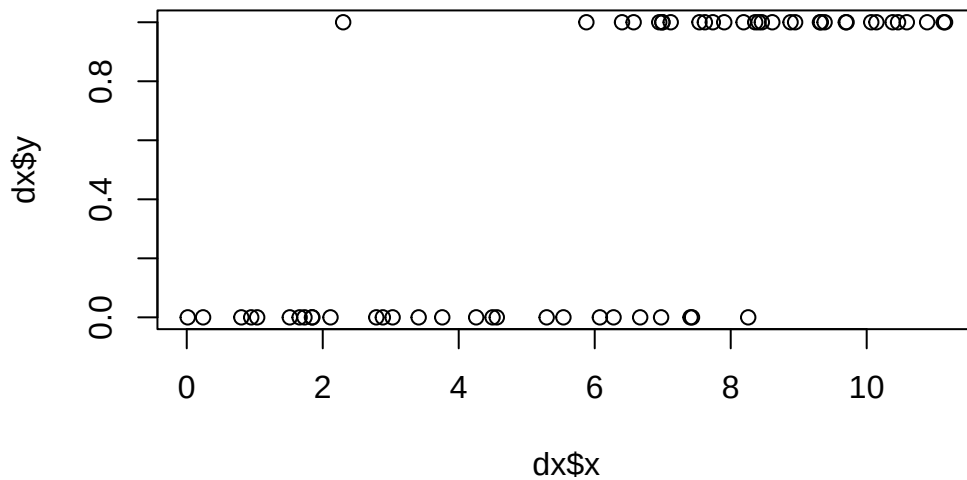


Figure 19: Plot of simulated binary response data, where $p(y=1)$ appears to increase with x .

We can fit logistic regression model to these data:

```
mod9 <- glm(y~x, data=dx, family=binomial(link="logit"))
summary(mod9)
```

Call:

```
glm(formula = y ~ x, family = binomial(link = "logit"), data = dx)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-5.9577	1.7244	-3.455	0.000550	***
x	0.9425	0.2471	3.815	0.000136	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 82.911 on 59 degrees of freedom

Residual deviance: 38.472 on 58 degrees of freedom
AIC: 42.472

Number of Fisher Scoring iterations: 6

The slope estimate for x , 0.9425, can be used to calculate the **odds ratio** or effect of x . Every unit increase in x multiplies the odds of $Y = 1$ by e^{β_1} . In this case, increasing x by 1 will multiply the odds of $Y = 1$ by $e^{0.9425} = 2.71282^{0.9425} = 2.5664$. Thus, negative slopes will decrease the likelihood of $Y = 1$, while positive slopes will increase it.

 Tip

Odds vs. probabilities

Odds and probabilities are two complementary ways of expressing how likely some event is. Probability is the number of successes per number of *trials*. The odds are the number of successes per number of *failures*. Odds and probability are related. E.g., if a coin comes up heads 2 times out of 8, then $p(\text{heads}) = 2/8 = 0.25$. The odds of heads are $(2/8) / (1 - (2/8)) = 0.25/0.75 = 1/3$.

Odds ratios are just one more step, expression a change in the probability as the change in odds. Odds ratios are particularly useful when discussing changes in probability near 0 or 1. E.g., changing probability from 0.96 to 0.98 or from 0.52 to 0.54 both have the same change in terms of probability (+0.02), but the first change is a much larger change in odds (OR = 2.04 vs. 1.08).

Unlike other GLMs, we don't use R^2 or pseudo- R^2 to evaluate logistic regression. Instead, we can measure its predictive accuracy. The hallmark of a well-fitting logistic regression model is that it will have very high **sensitivity** (true positive rate) and very high **specificity** (true negative rate). I.e., when the model predicts a 1 or 0, we can be very confident in that prediction being correct.

The **receiver operating characteristic (ROC) curve** summarizes the trade-off between sensitivity and specificity for a logistic regression model. The area under the curve, or **AUC**, is a value that ranges from about 0.5 (as accurate as a coin flip) to 1 (perfect sensitivity and specificity). It is available in the package **pROC**.

```
# install if needed
install.packages("pROC")

# load
library(pROC)
```

In order to get the AUC, we need to first get the model predictions (1s and 0s), and then compare that to the original data.

```
p9 <- predict(mod9, type="response")
roc9 <- roc(dx$y~p9, plot=TRUE, print.auc=TRUE)
```

Setting levels: control = 0, case = 1

Setting direction: controls < cases

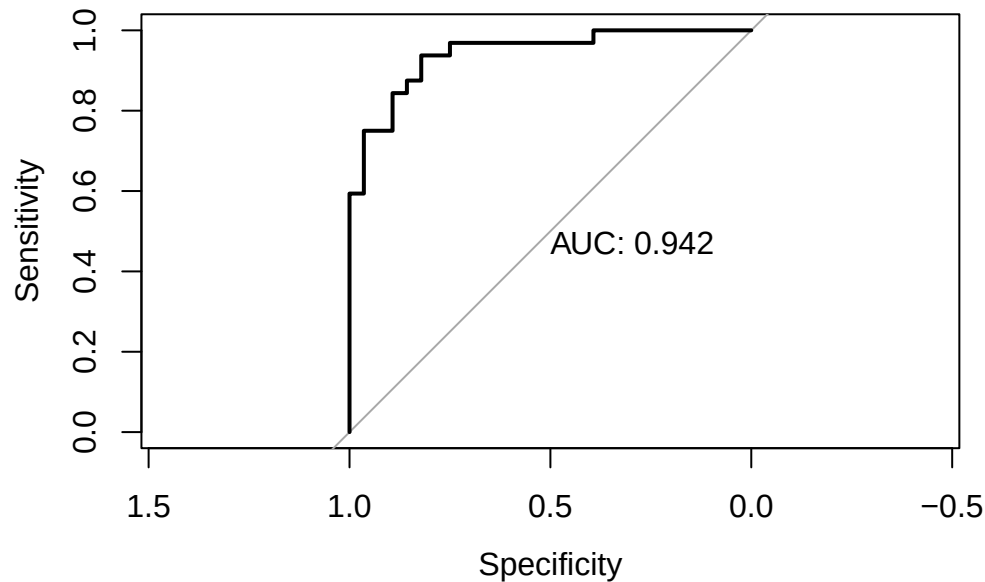


Figure 20: Receiver operating characteristic (ROC) curve for a logistic regression on simulated data. The area under the curve (AUC) is a measure of predictive accuracy.

The AUC of this model, 0.942, means that we can have high confidence in the model predictions.

Plotting a trendline for a logistic regression means plotting the data and a line for $p(Y = 1)$.

```
px <- seq(min(dx$x), max(dx$x), length=100)
prx <- data.frame(x=px)
pred <- predict(mod9, newdata=prx, type="link", se.fit=TRUE)
prx$mn <- plogis(qnorm(0.5, pred$fit, pred$se.fit))

plot(dx$x, dx$y, ylim=c(0, 1))
points(prx$x, prx$mn, type="l", lwd=3, col="red")
```

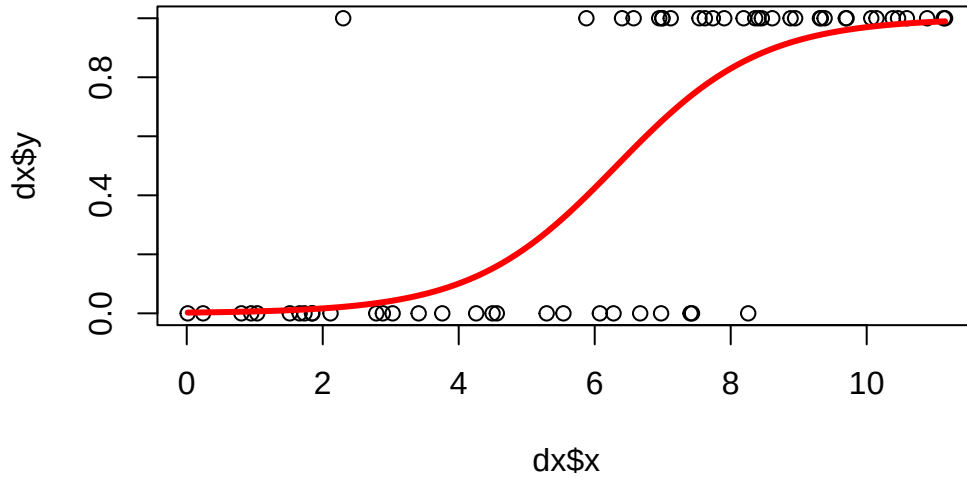


Figure 21: Plot of simulated binary response data, where $p(y=1)$ appears to increase with x , with red trendline predicted by the model.

8.3 Models for proportions: Binomial Regression

The last GLM that biologists encounter is **binomial regression**, which analyzes proportions or percentages. This is very close related to logistic regression in that they both use the **binomial distribution**. For a Y variable containing proportions or probabilities in the open interval $(0, 1)$:

$$Y_i = \frac{z_i}{n_i}$$

$$z_i \sim \text{Binomial}(n_i, p_i)$$

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 X_i$$

Because binomial and logistic regression are so similar, there is confusion about when to use each. The answer depends on your question:

- If the question is about outcomes at the individual observation level, then use logistic regression.
- If the question is about rates or probabilities of an event occurring, then use binomial regression.

```
set.seed(123)
n <- 50
dx <- data.frame(x=runif(n, 0, 10))
dx$z <- 3 + -0.75*x + rnorm(n, 0, 1)

# inverse logit
dx$y <- round(plogis(dx$z), 1)

# weights = sample sizes
# use same size for each observation for simplicity
dx$wts <- rep(10, n)

# plot of a proportion (y) vs. a predictor (x)
plot(dx$x, dx$y)
```

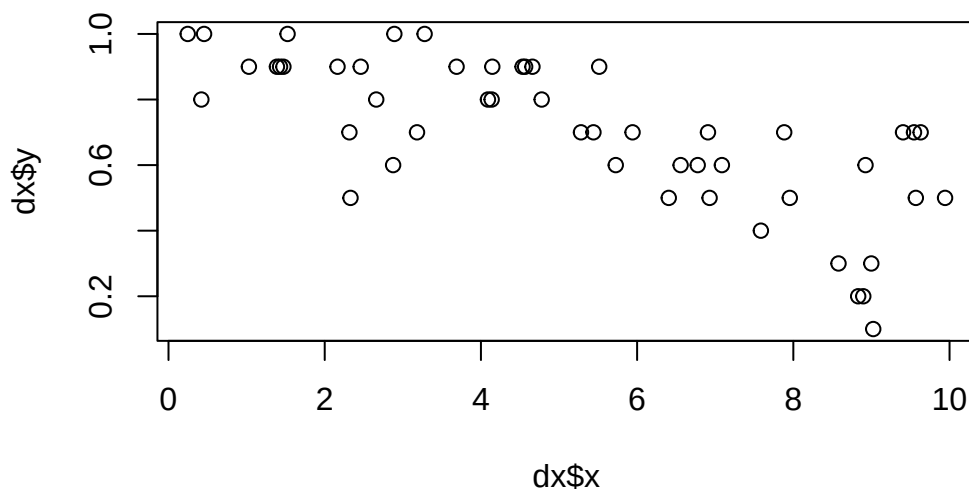


Figure 22: Plot of simulated binomial (proportion) response data, where y decreases as x increases.

Now we can fit a binomial GLM with the default logit link. The argument `weights` defines the sample sizes represented by each probability (Y value) converts the model to a binomial regression. For example, an observation with $Y = 0.3$ and `weights = 10` represents a trial or group of observations with 3 successes in 10 trials. You must supply a vector with a weight for each observation. Basically, if Y is a proportion, then `weights` is the denominator of that proportion.

```
mod10 <- glm(y~x, data=dx, family=binomial, weights=wts)
summary(mod10)
```

Call:

```
glm(formula = y ~ x, family = binomial, data = dx, weights = wts)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	2.51773	0.26011	9.679	< 2e-16 ***
x	-0.29588	0.03893	-7.600	2.96e-14 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 133.965 on 49 degrees of freedom
 Residual deviance: 66.704 on 48 degrees of freedom
 AIC: 180.02

Number of Fisher Scoring iterations: 4

Like Poisson regression, we can evaluate the model using the pseudo- R^2 and compare models using the AIC.

```
1-mod10$deviance/mod10$null.deviance
```

```
[1] 0.5020788
```

8.4 Gamma GLMs for overdispersed continuous data

Coming soon!

9 Tests for contingency

Sometimes biological data do not come with neatly-defined numeric responses. Often, the response “variable” is whether or not an observation belongs to a particular category or not—or a combination of categories. These can be called **categorical data**, where the information of interest is not a numeric value, but identity or classification. Consider the (made up) data from a pesticide exposure experiment below:

Table 1: Pollen production by pesticide treatment.

Pollen production	Unexposed	Exposed
None	17	6
Reduced	3	4
Normal	4	14

This table tallies the fates of 48 plants that were *exposed* or *not exposed* to an experimental pesticide. The outcomes were classified as *no pollen production*, *reduced pollen production*, and *normal pollen production*. Rather than asking a question about the mean pollen production in each group, the researchers might ask whether the fate of a plant (in terms of pollen production) is independent of whether or not it was treated. This is known as a **test of independence**.

A good clue for whether or not you need to do a test for independence is whether or not your data are best summarized in a contingency table such as the table above. A contingency table shows how often observations fall into each category within a dataset.

9.0.1 Fisher’s exact test

Fisher’s exact test is a test for independence in contingency tables that is usually only used when sample sizes are small. This is because the p -value calculation involves a numerous factorials, which can quickly overwhelm even modern desktop computers. Consequently, most software use an approximation to the true p -value, obviating the purpose of an exact test. For any situation where a Fisher’s exact test is appropriate, a chi-square (χ^2) test will usually give the same answer, but faster (as long as the sample size is large enough).

```
# make an R version of the matrix above
# make a matrix from the table above
a <- matrix(c(17,6,3,4,4,14), byrow=TRUE, nrow=3)
a
```

```
      [,1] [,2]
[1,]   17    6
```

```
[2,]    3    4
[3,]    4   14
```

And now the test:

```
fisher.test(a)
```

Fisher's Exact Test for Count Data

```
data:  a
p-value = 0.002849
alternative hypothesis: two.sided
```

Because Fisher's is an exact test (i.e., the p -value is an direct, exact calculation), there is no test statistic and no degrees of freedom to report.

9.0.2 Chi-squared test

The chi-square (χ^2) test is a more powerful, flexible, and computationally efficient cousin of the Fisher's exact test. It works by comparing the input contingency table to what the contingency table would be expected to look like under the assumption of independence. That is, if the null hypothesis were true. The test statistic, χ^2 , measures how different the two contingency tables are. The test works better with large sample sizes; with small sample sizes, the approximations use to estimate the p -value can break down. A good rule of thumb is that every cell (or most cells) of the hypothetical null contingency table should have ≥ 5 observations.

```
# make a matrix from the table above
a <- matrix(c(17,6,3,4,4,14), byrow=TRUE, nrow=3)

# run the test
chisq.test(a)
```

Warning in chisq.test(a): Chi-squared approximation may be incorrect

Pearson's Chi-squared test

```
data:  a
X-squared = 10.959, df = 2, p-value = 0.004171
```

One advantage of the χ^2 test in R vs. the Fisher's test is that it returns the expected contingency table, for comparison. This can help you see which categories are the most over- or underrepresented.

```
cq1 <- chisq.test(a)
```

Warning in chisq.test(a): Chi-squared approximation may be incorrect

```
cq1$observed
```

```
cq1$expected
```

```
      [,1] [,2]  
[1,] 11.5 11.5  
[2,]  3.5  3.5  
[3,]  9.0  9.0
```

This helps us see that the apparent lack of independence is likely driven by the relative numbers of “none” and “normal” pollen producers.